

AIDAP

AI Data Assurance Platform

Intended-use validation, policy gates, and evidence
for trusted AI inputs

WHITEPAPER

Version 1.0 | 17 September 2026

Validate what enters AI.

2. Information Page

Field	Publication information
Title	AIDAP - AI Data Assurance Platform
Version and date	1.0 • 17 September 2026
Author / organization	[Author name] / [Publishing organization]
Audience	Prospective customers, technical partners, investors, enterprise decision-makers, architects, compliance professionals, and AI and data teams
Purpose	Establish an evidence-based technical and commercial foundation for developing and commercializing AIDAP using the inspected SLAI codebase.
Scope	Input assurance for text, documents represented as text and metadata, structured datasets, retrieved context, and agent memory; product architecture, market position, economics, security, and delivery strategy.
Publication status	Professional stakeholder edition. Commercial prices and operating targets proposed for AIDAP are hypotheses, not offers or demonstrated performance.
Terminology	“Available” means an identifiable implementation is present in the source baseline. It does not mean independently tested, deployed, certified, or suitable for every input domain. “Proposed” identifies AIDAP design work.
Currency	AIDAP scenarios use EUR excluding VAT. Published competitor and model prices retain their original USD denomination. No exchange-rate equivalence is assumed.
Interpretation	This publication provides technical and commercial analysis, not legal advice, conformity assessment, certification, or a guarantee that any input or downstream system is safe or compliant.

2.1 Evidence basis and version boundary

The technical baseline is the SLAI repository at commit **265e3407a1deffe1d785215a67314b2da43b2ab9**, dated 29 August 2026, inspected on 17 September 2026. The related AIDAP repository snapshot is **1bcfee4f053d84bfa4c5e45a3b422d9913648533**. References identify the repositories and source paths supporting the capability assessment (The-Outsider-97, 2026a, 2026b).

“SLAI v2.3” denotes the requested product baseline anchored to that source snapshot. Internal version labels are not uniform: the Quality Agent declares 2.1.0, and agent-factory configuration includes a 2.2.0 default. A single, consistently tagged v2.3 release and a complete tested release manifest are not established by these labels. The capability conclusions therefore depend on the pinned implementation, not the version name alone. Release reconciliation is a commercialization prerequisite.

The evidence supports source-level statements about implemented paths, configuration, and interfaces. It does not establish end-to-end runtime success, production throughput, detector accuracy, security assurance, or customer adoption. No benchmark result or customer willingness-to-pay observation is asserted. This distinction prevents implementation presence from being mistaken for product readiness.

Public vendor documentation is treated as a vendor claim, not independent comparative testing. Market and regulatory observations are current to the publication date. Commercial estimates are explicitly identified as assumptions or recommendations. APA-style author-date citations connect factual statements to Section 10; source paths in Section 7.4 provide additional technical traceability.

3. Table of Contents

2. Information Page	2
2.1 Evidence basis and version boundary	2
3. Table of Contents	3
4. Executive Summary	5
5. Definitions	6
6. Introduction	8
7. Core Contents	9
7.1 AIDAP concept and product definition	9
7.2 Problem definition	10
7.3 AIDAP workflow	11
7.4 SLAI v2.3 architecture supporting AIDAP	13
7.5 Functionality requiring development outside SLAI	16
7.6 Technical architecture	18
7.7 Target audiences	20
7.8 User personas	22
7.9 Use cases	23
7.10 Competitive market analysis	24
7.11 Competitive positioning	27
7.12 Why SLAI instead of a simple LLM API wrapper	28
7.13 Business model	29
7.14 Cost model and unit economics	31
7.15 Scalability strategy	32
7.16 Market-entry strategy	33

7.17 Keeping AIDAP ahead of competitors 34

7.18 Product roadmap 35

7.19 Security and privacy 36

7.20 EU regulatory environment 37

7.21 Reliability and validation strategy 38

7.22 Risks and limitations 39

7.23 Commercial KPIs 40

7.24 Long-term strategy 41

8. Conclusion 42

9. Recommendations 43

10. References 44

4. Executive Summary

AIDAP is a proposed commercial **AI Data Assurance Platform** governed by the principle **Validate what enters AI**. It evaluates an input version against a declared use, destination, and policy, then returns an auditable pass, review, quarantine, or block decision. Its output is a controlled release decision with evidence and explicit limitations, not a guarantee of downstream correctness.

The initial product should protect RAG and AI-software ingestion. Before indexing a knowledge update, it would check structure, metadata, freshness, duplication, configured consistency, privacy restrictions, and provenance. Evidence would identify the input, policy, coverage, and authorized exceptions. Selected SLAI implementations supply a validation kernel; AIDAP must supply the service and enforcement boundary.

Input assurance addresses an important failure mode: a model can produce fluent output from stale, contradictory, unauthorized, or irrelevant information. Retrieval improves access to external knowledge but does not make that knowledge trustworthy (Lewis et al., 2020). Data quality is also contextual: data that is acceptable for one task can be unsuitable for another (Wang & Strong, 1996). Model-side guardrails and output evaluation remain necessary, but cannot retrospectively prevent an unauthorized disclosure that occurred when information was submitted to a model.

The market gap is narrow. Data observability vendors are expanding into AI trust and unstructured data; evaluation vendors offer inexpensive entry tiers and preventive controls. Fiddler's control-plane positioning and Galileo's guardrails are direct threats (Fiddler, n.d.-a; Galileo, n.d.). AIDAP should compete through affordable task-specific decisions, evidence, customer-controlled processing, and low integration effort. Pre-inference screening alone is not unique.

SLAI provides structural/statistical checks, configured semantic rules, privacy and safety components, and reusable execution, memory, configuration, and checkpoint machinery. Its semantic-quality path is not a general evaluator of document truth or RAG sufficiency. Typed failure handling, bounded execution, persistence, isolation, and integration testing are required before production gating (The-Outsider-97, 2026a). This publication anchors "v2.3" to an inspected source commit because internal version labels differ.

The beachhead is engineering-led software vendors repeatedly updating RAG knowledge bases. AI consultancies offer a potential distribution channel. Regulated enterprises are later opportunities requiring stronger domain validation, security evidence, and delivery capacity. Recurring ingestion work, rather than one-time cleanup, is essential to the revenue thesis.

Combine a free developer allowance with predictable subscriptions and capped usage. Test €39 Starter, €149 Team, and €499 Business monthly, with separate model allowances and prepaid extensions. These are hypotheses, not validated willingness to pay. Private deployments require annual minimums and scoped onboarding. An open SDK supports adoption; hosted policy, evidence, review, and operations earn recurring revenue.

Affordability depends on deterministic-first checks, selective inference, safe caching, asynchronous processing, and bounded workers. Start with one application service, PostgreSQL, object storage, and a queue. Defer distributed agent negotiation, continuous retraining, broad connectors, and a general governance suite.

Principal risks are false confidence, imperfect semantic judgments, hardening effort, support-heavy accounts, and incumbent bundling. Prove release integrity and bounded detectors, run paid pilots, establish contribution margin, then expand task-specific profiles. Sustainable advantage must come from validated policies, permitted feedback, integration reliability, and accumulated evidence rather than agent count or API access.

5. Definitions

The following terms describe the product and its operating model. A decision always relates to a named input version, policy version, and intended use; it is not a universal property of the data.

Term	Definition
AIDAP	AI Data Assurance Platform: the proposed product that applies validation and policy to information before approved downstream use.
SLAI	Safe Learning AI: the modular agent framework and implementations used as AIDAP's proposed validation kernel. The inspected baseline is specified in Section 2.1.
AI Data Assurance	A documented process for assessing and controlling whether data meets specified requirements for an AI-related use, with evidence and explicit uncertainty.
Data Quality	The degree to which data meets relevant intrinsic, representational, accessibility, and contextual requirements.
Data Fitness	Suitability of data for a specified purpose, population, time horizon, and operating context.
AI Input Fitness	Suitability of the exact information supplied to a particular AI task, including quality, relevance, permissions, and risk constraints.
AI Input Assurance	Controls and evidence applied at the boundary where information is admitted to an AI workflow.
Data Observability	Monitoring and diagnosis of data-system behavior and data health through signals such as freshness, volume, schema, distributions, and lineage.
AI Observability	Collection and analysis of AI-system behavior, including traces, inputs, outputs, costs, latency, and evaluation results.
RAG	Retrieval-augmented generation: generation conditioned on information retrieved from an external knowledge source.
Agentic AI	Software that uses AI to select or execute steps toward goals, potentially using tools, memory, and external systems.
Quality Gate	An enforceable decision point that permits, holds, isolates, or rejects information according to explicit requirements.
Validation	Checking conformance to specified requirements, including schema, policy, or task constraints.
Evaluation	Measuring behavior or quality against criteria, examples, or reference judgments; it may inform validation without itself enforcing a gate.
Structural Validation	Checking types, schema, required fields, formats, integrity constraints, and representation.
Statistical Validation	Assessing properties of populations or batches, including missingness, distributions, outliers, duplication, and change relative to baselines.
Semantic Validation	Assessing meaning-related consistency or suitability. It may use explicit rules, statistical models, or language models; the term does not imply factual verification.
Privacy Screening	Detection and policy assessment of personal, sensitive, or restricted information and its permitted use or destination.
Safety Screening	Assessment of content or instructions against specified harm, security, and operational-risk policies.
Pass	Permission to release the evaluated input version for the specified use, subject to expiry and destination constraints.
Review	A hold pending an authorized judgment about an ambiguous or exception-eligible finding. No downstream release is implied.

Term	Definition
Quarantine	Isolation of a retained artifact from ordinary processing while investigation, repair, or disposal proceeds.
Block	Rejection of the requested use under the applicable policy. Retention or deletion is governed separately.
Evidence Bundle	A versioned record linking input identity, policy, evaluator results, coverage, decision, and relevant review or release events.
Provenance	Information describing origin, transformations, responsible actors, and derivation relationships. Provenance does not establish truth by itself.
Data Drift	Change in input-data distributions or properties relative to a defined reference. Drift can be benign or harmful.
Model Drift	Degradation or change in model behavior or performance over time; it is distinct from observing only input-distribution change.
Data Contract	An explicit agreement specifying data structure, semantics, ownership, service expectations, and permitted change.
Human-in-the-loop	A workflow in which authorized people supply judgments or approvals at defined stages, with their interventions recorded.
Deterministic validation	A check intended to yield the same result from the same input, rule, and execution environment.
Probabilistic validation	A check whose conclusion depends on statistical inference, learned models, sampling, or stochastic computation.
Confidence	An uncertainty estimate with a declared interpretation. A normalized heuristic score is not automatically a calibrated probability of correctness.
False positive	A finding that flags a condition as present when the reference judgment says it is absent.
False negative	A missed condition that the reference judgment says is present.
Multi-tenancy	Operation of one service for multiple customers with explicit isolation of identity, access, data, state, and resource use.
Self-hosting	Customer operation of the application within infrastructure the customer controls.
SaaS	Software as a service: a provider-operated application accessed through a service interface.
Usage-based pricing	Charges that vary with measured consumption under a published meter and billing policy.
Idempotency	The property that repeating a request with the same identity does not cause unintended duplicate effects.
Coverage	The checks and portions of an input actually evaluated, including exclusions, sampling, and unsupported content.
Fail closed	Withholding release when a mandatory check cannot complete or returns an invalid result. It does not necessarily require permanent rejection.
ARR / MRR	Annual / monthly recurring revenue, excluding one-time services and non-recurring pass-through charges under the chosen accounting definition.
Gross margin	Revenue less direct service-delivery cost, divided by revenue; distinct from operating profitability.
CAC / LTV / NRR	Customer acquisition cost; estimated customer lifetime value; and net revenue retention from an existing customer cohort, respectively.

6. Introduction

AI applications increasingly assemble their effective inputs from documents, database records, retrieved passages, tool responses, uploaded files, and persistent memory. The model sees a selected and transformed view of these sources, often without the human context that originally qualified their meaning. A superseded policy can look authoritative; an empty field can be interpreted as a negative answer; an injected instruction can be mistaken for a legitimate task requirement.

This is a modern form of the “garbage in, garbage out” problem, with an additional difficulty: fluent output can conceal defective inputs. RAG combines a generative model with retrieved knowledge, but the retrieval corpus, ranking, access controls, and context assembly remain independent sources of failure (Lewis et al., 2020). Data documentation can make intended uses, collection conditions, and limitations explicit, yet documentation must be connected to operational checks to influence what is admitted (Gebru et al., 2021).

Output-quality monitoring helps identify errors after execution. Input assurance controls exposure before execution. These are complementary functions. For example, an output detector may suppress an answer containing a personal identifier, but a disclosure to an external model provider may already have occurred in the request. Similarly, a retrieval metric may reveal poor relevance while leaving the decision to index an outdated or unauthorized document uncontrolled.

Model-side instructions alone are insufficient for a dependable boundary. Retrieved content can carry adversarial instructions, as demonstrated by research on indirect prompt injection (Greshake et al., 2023). A gate should treat the payload as untrusted data, limit evaluator privileges, and apply explicit permissions and release rules independently of the evaluated text. No detector eliminates the need for access control, least privilege, and downstream safety controls.

The basic commercial architecture is **Source Data → AIDAP → AI / RAG / Agent / ML / Analytics System**. The intermediary earns its place only if it makes a concrete decision easier and more reliable: whether a particular input may proceed, what must be repaired, and what evidence explains the outcome. It should not require customers to replace their data platform, vector database, tracing system, or existing quality framework.

The strongest commercial rationale is reduced operational friction in recurring input changes. A customer pays for fewer avoidable ingestion failures, shorter reviews, clearer ownership of exceptions, and reproducible release records. A claim to eliminate hallucinations or guarantee compliance would exceed both the technical evidence and a sensible product boundary. AIDAP should make the limits of an assurance decision visible and actionable.

7. Core Contents

7.1 AIDAP concept and product definition

AIDAP should be an API-first control layer with a compact review interface. Its unit of work is an **assurance request**: an immutable input or input manifest, intended-use profile, destination, policy version, and execution budget. Its primary output is a structured decision plus evidence. Validation is useful only when the consuming integration respects the decision and cannot silently bypass it.

An intended-use profile should specify the task, permitted audience, source classes, freshness window, required metadata, applicable privacy restrictions, required checks, acceptable uncertainty, and release destination. A public product-support assistant may accept published manuals but reject internal incident notes. An internal analytical task may permit pseudonymized records that remain inappropriate for a public external-model endpoint. The same artifact can therefore receive different decisions for different uses without inconsistency.

Table 1. Proposed AIDAP decision semantics. SLAI's pass/warn/block and privacy allow/modify results require explicit mapping; they do not already implement this complete product state machine.

Decision	Operational effect	Resolution and evidence
Pass	Release the exact approved artifact or sanitized derivative to an authorized destination.	Record content digest, policy/evaluator versions, coverage, expiry, and release identity. Recheck material changes.
Review	Hold release and create an assigned review item.	An authorized reviewer approves an exception, rejects, or requests repair; reason and scope are recorded. Timeout remains a hold.
Quarantine	Place the artifact in restricted storage, remove it from normal indexing or delivery, and preserve investigation evidence.	Require explicit remediation and revalidation; quarantine access is distinct from ordinary review permission.
Block	Reject the requested operation.	Return machine-readable reasons. A repaired input or policy-authorized exception creates a new decision, rather than rewriting history.

Review and quarantine are not interchangeable: review is a decision workflow; quarantine is an isolation action. A blocked malicious file may also be quarantined if retention is justified. A review item need not preserve full raw content. Internally, AIDAP should model decision status, artifact disposition, and job state separately to avoid ambiguous transitions.

The product is not a chatbot, vector store, warehouse monitor, autonomous remediation agent, or general LLM benchmark suite. It should consume existing quality findings where appropriate and export decisions into existing pipelines. It is also not an authority on legal truth or domain correctness. “Fit” means that specified requirements were satisfied within declared coverage, not that all possible defects have been excluded.

7.2 Problem definition

Input failures span several layers. A structurally valid document can still be stale; a statistically ordinary record can still be unauthorized; a relevant passage can still contradict the controlling policy. AIDAP should expose these dimensions separately so that a good score on one cannot compensate for a prohibited condition on another.

Table 2. Input failure taxonomy and corresponding controls. Responses are proposed product controls, not claims that every detector already exists in SLAI.

Failure class	Representative defect and consequence	Appropriate response
Structural integrity	Schema mismatch, missing mandatory metadata, invalid units, or inconsistent values causes broken transforms or misleading features.	Deterministic validation; hold or block mandatory violations.
Population quality	Duplicates, missingness, extreme values, and changed distributions bias retrieval or analytics and increase debugging effort.	Profile batches against approved baselines; assess change significance and task impact.
Temporal validity	Superseded instructions or stale feed records produce outdated recommendations.	Enforce effective dates, expiry, source update expectations, and revision relationships.
Contextual fitness	Irrelevant passages, insufficient evidence, or incompatible document scopes produce unsupported answers.	Task-specific checks; abstention or review when coverage is inadequate.
Semantic consistency	Conflicting policy versions or cross-field contradictions undermine decisions.	Configured rules first; bounded semantic comparisons with cited spans later.
Privacy and authorization	Personal data, secrets, or restricted corporate material reaches an inappropriate model or audience.	Destination-aware permissions and screening before any external evaluation.
Adversarial inputs	Poisoned content, embedded instructions, or parser exploits influence tools or evaluators.	Isolation, safe parsing, untrusted-content handling, and layered security.
Provenance uncertainty	Unknown source, missing owner, or unverified transformations prevents responsible use.	Require provenance metadata; distinguish asserted from independently verified origin.

Financial exposure is highly task-dependent. It can include engineer time, repeated model calls, support escalations, re-indexing, delayed deployment, loss of trust, and incidents involving privacy or unsafe decisions. There is no defensible universal cost-per-bad-record figure. Sales discovery should quantify the customer's own recent failures and the proportion plausibly preventable by the proposed checks.

Statistical unusualness is not itself a defect. Seasonal sales, a new product launch, and a deliberate policy change can all create drift. Anomaly research offers methods for identifying departures from learned patterns, but the mapping from anomaly to business harm remains contextual (Chalapathy & Chawla, 2019). A useful gate therefore separates observation, interpretation, and policy consequence.

For RAG, prevention has two locations: before indexing, where source eligibility and document quality are assessed; and after retrieval, where the selected context is assessed for a particular query. The first is a feasible initial product. The second requires additional task-aware evaluation and latency engineering and should not be implied by the existence of a semantic-quality module.

7.3 AIDAP workflow

The workflow below is a proposed product pipeline. “D” denotes deterministic logic; “S” statistical inference; “M” optional learned/model-assisted evaluation; and “H” authorized human judgment. A deterministic implementation can still encode an incorrect rule, while a probabilistic detector can still be useful when its error profile is measured.

Table 3. End-to-end workflow and computational method. The sequence is logical, not a mandate to defer privacy until step 9: preliminary local privacy and security screening must run before step 8 whenever that step uses an external provider.

Stage	Execution and output	Method and implementation boundary
1. Ingestion	Authenticate source; bind tenant, asset, destination, and idempotency key; impose size limits.	D; AIDAP API and connectors outside SLAI.
2. Normalization	Preserve original digest; decode supported formats; create canonical records or text plus metadata.	D; narrow adapters around SLAI-compatible inputs. Hostile-file parsing needs isolation.
3. Format/schema validation	Check schema version, types, fields, allowed formats, and parse coverage.	D; StructuralQuality reuse plus adapters.
4. Structural quality	Apply requiredness, ranges, domains, and integrity rules.	D; existing quality subsystem, with domain configuration.
5. Statistical quality	Measure missingness, duplicates, distributions, and sample coverage.	D/S; StatisticalQuality reuse and approved baseline handling.
6. Anomaly detection	Compare numeric/categorical properties with relevant references.	S; existing checks are bounded by supported data and assumptions.
7. Relationship analysis	Check configured cross-field dependencies and reference keys.	D; limited existing consistency rules; broad cross-asset joins need development.
8. Semantic fitness	Apply configured meaning-related rules; optionally test relevance or contradiction.	D/M; current rules exist; general RAG/document evaluators require substantial work.
9. Privacy screening	Identify restricted data and evaluate purpose, destination, and transformation requirements.	D/M; Privacy Agent components plus product permissions. Must precede external model calls.
10. Safety screening	Apply selected threat and content checks to untrusted inputs.	D/M; selected Safety Agent components need domain tests.
11. Provenance/evidence	Check source identity, revision metadata, ownership, and transformation chain.	D; existing metadata assessment plus AIDAP evidence store.
12. Intended-use fitness	Compare findings and coverage against the named task profile.	D/M; AIDAP policy composition; task-specific semantics are new work.
13. Confidence	Record calibrated estimates where available and explicitly mark heuristic/unknown values.	S/M; calibration requires labeled data; no invented certainty.
14. Policy decision	Apply mandatory constraints, severity, unresolved checks, and exception rules.	D; authoritative AIDAP decision engine.
15. Disposition	Pass, hold for review, quarantine, or block; keep job state separate.	D/H; product state machine and review permissions.
16. Evidence generation	Persist findings, versions, coverage, digests, and rationale.	D; reuse subsystem outputs, normalize outside SLAI.
17. Audit logging	Record actors and state transitions with correlation identifiers and restricted payloads.	D; durable product audit service.

Stage	Execution and output	Method and implementation boundary
18. Checkpointing	Commit completed stages and restart-safe state before retries or delivery.	D; SLAI checkpoints plus AIDAP job ledger.
19. Alerting	Notify assigned owners, deduplicate repeated incidents, track acknowledgment.	D; product events, email/webhooks, later ticketing integrations.
20. Delivery	Release only the approved digest to the approved destination using an idempotent outbox.	D; connector verifies decision and expiry; AIDAP integration work.

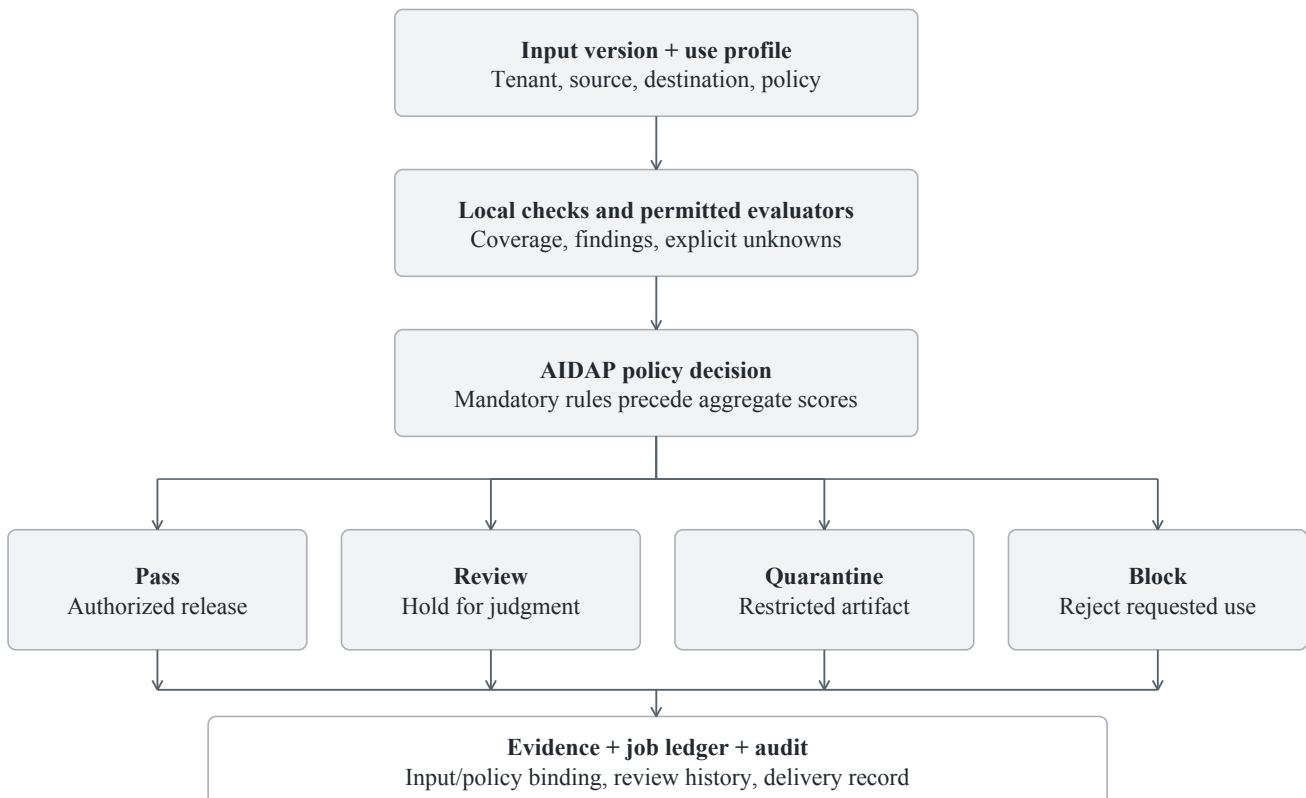


Figure 1. Proposed intake decision and release workflow. An unknown mandatory result cannot become pass. Human approval creates an attributable exception or a new validated version.

7.3.1 Decision logic and confidence

The decision engine should apply ordered policy rules. A prohibited destination or confirmed mandatory violation blocks the requested operation. Suspicious artifacts requiring isolation are quarantined. Incomplete mandatory checks, ambiguous semantic findings, or permissible exceptions enter review. Pass requires all mandatory checks to finish acceptably and any required reviewer authorization to be present. Optional check failures remain visible in coverage and cannot be silently described as successful validation.

A weighted quality score can help sort work, but must not override a privacy or safety constraint. SLAI's configured quality thresholds and subsystem weights are useful starting parameters, not empirically calibrated probabilities. Calibration requires held-out examples and an explicit target event, such as agreement with adjudicated document eligibility. Modern learned models can be miscalibrated; score interpretation must be tested rather than assumed (Guo et al., 2017).

Every finding should distinguish observed evidence, detector inference, confidence interpretation, severity, and recommended action. A contradiction detector, for example, should cite the two spans and their scopes; it should

not invent a factual explanation without evidence. Where a check is unsupported, unavailable, or skipped for budget reasons, the result is unknown or not evaluated, not a perfect score.

7.3.2 Evidence contract and example

The minimum evidence schema should include tenant and run IDs; asset and source version; original and normalized digests; intended-use and destination IDs; policy version; evaluator code/model/prompt versions; timestamps; baseline ID; check coverage; findings and redacted evidence pointers; decision; reason codes; reviewer/exception identity; expiry; resource usage; and delivery status. A cryptographic digest establishes identity and change detection, not authenticity or factual accuracy. Provenance should distinguish entities, activities, and agents in a manner compatible with W3C PROV concepts (W3C, 2013).

Consider an uploaded product manual with a valid schema, a superseded effective date, and an embedded employee email address. Structural validation can pass while freshness fails and privacy requires redaction. AIDAP should hold the document, present the precise findings, and avoid indexing the original. An owner can supply the current manual or authorize an appropriate restricted use. The sanitized derivative receives a new digest and is rechecked; it does not inherit approval merely because a transformation was attempted.

Checkpointing must preserve the binding between input, policy, and stage outputs. A worker that restarts after assessment but before delivery should not re-charge a completed stage or publish twice. A transactional outbox and destination idempotency are product-level requirements; an agent checkpoint alone cannot provide exactly-once delivery across external systems.

7.4 SLAI v2.3 architecture supporting AIDAP

7.4.1 Capability classification

Five classes are used throughout this analysis: **A** - **available implementation**, **H** - **existing implementation requiring production hardening**, **I** - **minor integration around an existing capability**, **N** - **substantial new development outside SLAI**, and **O** - **deliberately outside SLAI's responsibility**. A/H may appear together: a function can exist while remaining unsuitable for unqualified production claims. I is a relative engineering classification, not a guaranteed schedule.

The related AIDAP repository currently provides a substantial architectural README and a largely skeletal product layout. Empty or near-empty bootstrap, package, and frontend files do not establish an operating SaaS product. Its separation of domain, contracts, engine, ports, adapters, API, and workers is a sound architectural starting point, but should be described as a scaffold (The-Outsider-97, 2026b).

Table 4. Source-grounded component assessment. Source: The-Outsider-97 (2026a), pinned baseline. “Available” denotes implementation presence, not production certification.

Component	Present role and evidence	AIDAP use, maturity, and limits
Base Agent	Lifecycle, execution dispatch, recovery hooks, health/state machinery, and checkpoint integration in <code>base_agent.py</code> .	A/H. Reuse a narrow, explicit execution path. Typed failures, retry behavior, time limits, and lifecycle side effects need hardening.
Agent Factory	Registration, construction, configuration, and lifecycle facilities in <code>agent_factory.py</code> .	A/H/I. Instantiate an allowlist of needed agents. Factory abstractions do not prove secure tenant/process isolation.
Orchestration	Quality Agent and collaborative framework coordinate components and shared artifacts.	A/H. Use a fixed AIDAP stage plan; do not require autonomous agent negotiation for routine checks.

Component	Present role and evidence	AIDAP use, maturity, and limits
Quality Agent	Coordinates normalization, structural/statistical/semantic checks and workflow control. Produces reports and pass/warn/block-style outcomes.	A/H/I. Best initial kernel. Map its results into AIDAP's explicit four-decision contract; preserve subsystem failures and coverage.
StructuralQuality	Required fields, types, ranges/domains, coercion, and schema consistency over supported records.	A/H/I. Useful deterministic core. Text documents need metadata/text adapters; coercion must preserve the original and be disclosed.
StatisticalQuality	Missingness, duplicates, numeric outliers, and numeric/categorical baseline comparisons.	A/H. Useful batch checks. Approve baselines and validate small-sample behavior; no universal anomaly accuracy follows.
SemanticQuality	Label-leakage heuristics, configured cross-field consistency, and provenance assessment.	A/H for these functions. General document contradiction, relevance, and context sufficiency are N, not existing broad semantic intelligence.
Quality memory/workflow	JSON-backed quality state/journal, schema and threshold records, routing/quarantine-related artifacts.	A/H. Local coordination is useful; secure quarantine storage, durable multi-worker transactions, and tenant boundaries are N/O.
Privacy Agent	Identification, consent/purpose checks, minimization, retention-related processing, audit/evidence construction.	A/H/I. Reuse supported detectors and transformations. Legal basis, destinations, actual storage deletion, and regional controls belong to AIDAP.
Safety Agent	Coordinates configurable safety, threat, sanitization, cyber-risk, reward, and STPA-related components.	A/H. Select only input-relevant checks. Domain coverage and prompt-injection efficacy require testing; action-safety components are not automatically document validators.
Evaluation Agent	Broad code/system evaluation, issue scoring, and an IsolationForest path over issue features.	A/H, restricted applicability. Its training-data loader includes two hardcoded example issue records. It is not a trained general AI-input anomaly service. Exclude full agent from MVP default path.
Reasoning Agent	Structured facts, knowledge/rule inference, and reasoning traces.	A/H. Optional later use for explicit domain rules. It does not establish general factual truth or automatic contradiction resolution across arbitrary documents.
Execution Agent	Tool/action execution capabilities.	A; deliberately unused in the read-only MVP. Approved delivery should be a narrow product connector, not unconstrained agent action.
Adaptive / Learning Agents	Skill routing, reinforcement/meta-learning, recovery, and training-related machinery.	A/H. Keep outside live policy changes. Offline experimentation may be useful; useful trained evaluators and safe continuous learning are not established.
Shared memory	Process-resident shared state, version history, TTL, locking, subscribers, and save/load facilities.	A/H. Transient execution context only. It is not a distributed database or a security boundary between customers.
Checkpoint manager	Codec/component validation, storage policy, manifests, and agent state restoration machinery.	A/H/I. Existing implementation should be exercised with real failure/recovery tests. Product release and billing checkpoints remain separate.

Component	Present role and evidence	AIDAP use, maturity, and limits
Tuning / configuration	Configurable agents, reusable functions, and offline tuning/evaluator machinery.	A/H. Pin configurations; test candidate thresholds offline. Configuration availability is not evidence of calibrated performance.
Data processing / Reader	Parsing and data-handling components exist.	A/H/I for narrowly supported inputs. General file ingestion, OCR, sandboxing, and robust format coverage are N.
Logging / transport / auth	Reusable logging and transport/auth functions exist.	A/H/I as utilities. They do not constitute an integrated customer API, RBAC, tenant model, billing system, or secure audit platform.
Model independence	Many useful checks run locally without a foundation-model API; adapters can separate model-dependent work.	A/I for deterministic independence. Interchangeable semantic quality across every model/backend is not established.

7.4.2 Findings that materially affect production design

Quality Agent defaults include stopping on blocking structural findings and failing closed on subsystem error. Its default pass/warn thresholds are 0.9 and 0.75, with structural/statistical/semantic weights approximately 0.34/0.33/0.33. These demonstrate a configurable decision mechanism, not an empirically justified operating point. AIDAP must validate the mapping against its own policy contract and must not allow averaging to erase a mandatory prohibition.

The statistical subsystem can register a baseline when none exists. That convenience is unsuitable as implicit approval of the first customer batch: the first batch may be defective or adversarial. AIDAP should mark an absent baseline explicitly, separate baseline creation from approval, and version baseline changes. Drift comparisons need comparable schema, population, sampling, and time window.

The semantic subsystem's named capabilities are narrower than the product ambition. Its inspected path calls label-leakage detection, cross-field consistency validation, and provenance assessment, using configured rules and heuristics such as string similarity. It does not provide a general foundation-model or embedding-based RAG-fitness evaluator on that path. Broad contextual fitness must therefore be designed, integrated, and validated separately.

Privacy processing is more than a placeholder: it identifies information, checks consent/purpose/action/destination conditions, applies minimization, and builds retention and audit-related results. Failure paths in consent checking can raise errors. Nevertheless, a consent-oriented implementation cannot by itself determine the correct legal basis for every processing activity. Nor can a retention method prove deletion from product databases, object versions, backups, exported files, or external providers without corresponding integrations.

Base Agent requires particular attention. Its execution path can treat a returned value as successful without a strict product-level result contract. A post-success hook failure can enter retry behavior, and generic fallback output must not be interpreted as completed assurance. Flexible capability dispatch and exception handling can obscure whether a detector actually ran. The presence of timeout configuration does not itself demonstrate an enforced execution deadline. AIDAP should call explicit typed adapters, reject malformed results, impose worker-level time and memory limits, and separate evaluation from side effects.

Checkpoint implementation is present in the inspected current source; it should not be dismissed as a stub on the basis of older descriptions. The remaining question is operational correctness: complete state capture, schema compatibility, storage integrity, recovery behavior, and safe handling of untrusted serialized content. Persisting an agent's state does not establish an atomic relationship among decision, audit record, billing meter, and downstream release.

7.4.3 Source traceability and reuse boundary

Table 5. Technical evidence map. Repository-level references provide the immutable baseline. Paths identify evidence locations; file names alone are not evidence of end-to-end capability.

Capability or finding	Principal source paths within the pinned SLAI repository
Execution, dispatch, lifecycle, checkpoints	src/agents/base_agent.py; checkpointing/checkpoint_manager.py
Construction and version configuration	src/agents/agent_factory.py; src/agents/base/configs/agents_config.yaml
Quality aggregation and workflow	src/agents/quality_agent.py; src/agents/quality/workflow_control.py
Structural and statistical checks	src/agents/quality/structural_quality.py; src/agents/quality/statistical_quality.py
Semantic scope and quality state	src/agents/quality/semantic_quality.py; src/agents/quality/quality_memory.py
Privacy and retention	src/agents/privacy_agent.py; src/agents/privacy/data_id.py; data_consent.py; data_minimization.py; data_retention.py in the same directory
Safety and evaluation	src/agents/safety_agent.py; src/agents/evaluation_agent.py
Reasoning, execution, adaptation, learning	src/agents/reasoning_agent.py; execution_agent.py; adaptive_agent.py; learning_agent.py in src/agents/
Shared state, parsing, utilities	src/agents/collaborative/shared_memory.py; src/agents/reader_agent.py; src/functions/transport.py; src/functions/auth.py
Offline tuning	src/tuning/README.md and the corresponding tuning package

The practical reuse boundary is a small, pinned SLAI dependency behind AIDAP-owned contracts. AIDAP should own the policy schema, four decisions, evidence schema, review state, tenancy, release authorization, and metering. Agents should return findings, not directly modify customer authorization or deliver data. This preserves the option to improve or replace a validator without rewriting the commercial product.

7.5 Functionality requiring development outside SLAI

An application framework is not a commercial operating environment. Keeping product infrastructure outside SLAI reduces framework coupling and makes customer responsibilities, security boundaries, and deployment upgrades easier to reason about. The AIDAP domain layer should not import agent implementations; adapters should translate between stable product contracts and the pinned SLAI interfaces.

Table 6. Development outside the SLAI kernel. A small integration is small only when its contract, failure behavior, and input domain are tightly bounded.

Product capability	Classification	Required implementation and rationale
Frontend and review console	N/O	Build compact runs, findings, policy, review, and usage views. This is a product workflow, not an agent responsibility.
Backend API and SDK	N/O; individual adapters I	Versioned endpoints, typed errors, idempotency, pagination, upload limits, and Python SDK. Existing transport utilities do not replace this service.

Product capability	Classification	Required implementation and rationale
Authentication, authorization, RBAC	N/O	Use a maintained identity provider; enforce tenant/project roles and service-account scopes server-side.
Tenant management	N/O	Tenant-owned IDs, database isolation rules, storage prefixes/keys, quotas, secret boundaries, and deletion workflows.
Persistent database	N/O	PostgreSQL for policies, jobs, decisions, usage, review, and outbox records; migrations and transaction design.
Object storage and quarantine	N/O	Encrypted artifacts/evidence, restricted quarantine access, lifecycle rules, signed URLs, and deletion verification.
Queue, workers, cache	N/O	Durable scheduling, bounded concurrency, retries, dead-letter handling, and tenant-aware cache keys.
Cloud/container operations	N/O	Minimal images, secrets, networking, infrastructure as code, CI/CD, patching, backups, and restoration.
Observability and audit	N/O	Service metrics/traces separated from immutable decision events; payload redaction and access controls.
Billing, subscriptions, VAT/tax	N/O	Meter ledger, plan entitlements, invoices, tax handling, dunning, credits, cancellation, and spend controls. Buy commodity billing infrastructure.
Connectors, webhooks, integrations	I for one simple adapter; N for a supported catalog	Signed delivery, credential lifecycle, retries, version compatibility, warehouse/RAG/CI integrations, and documented support scope.
Model APIs and local inference	I for a narrow adapter; N for robust fleet operation	Provider contracts, routing, budgets, residency, model/version pinning, and local-serving operations.
Advanced task-specific semantics	N	Labeled datasets, evaluators, calibrated thresholds, retrieval/context logic, and error analysis beyond current SLAI rules.
Governance, legal determinations, certification	O	Product evidence may assist these processes. Organizational accountability and legal/conformity judgments do not belong inside an agent.

Recommended API resources are projects, assets, input versions, profiles, policies, runs, findings, decisions, reviews, and deliveries. A run should be submitted asynchronously and polled or completed by signed webhook. A future synchronous gate can reuse the same decision contract but should have a separate latency and availability envelope.

The initial connector set should be one generic upload/API path and one object-store or RAG ingestion example. Warehouse scanning, managed OAuth connectors, OCR, and unrestricted web crawling increase operational and security burden materially. They should be justified by paid demand, not by the desire to match incumbent feature lists.

7.6 Technical architecture

7.6.1 Production service design

The recommended first production architecture is a modular monolith with separately scalable workers. A stateless API authenticates requests, checks entitlements, and creates a job and outbox event in PostgreSQL. Uploaded content is stored under a tenant-bound immutable object identity. Workers retrieve a signed job specification, run a fixed validation plan through SLAI adapters, and commit normalized findings and the decision. Delivery occurs only after that commit.

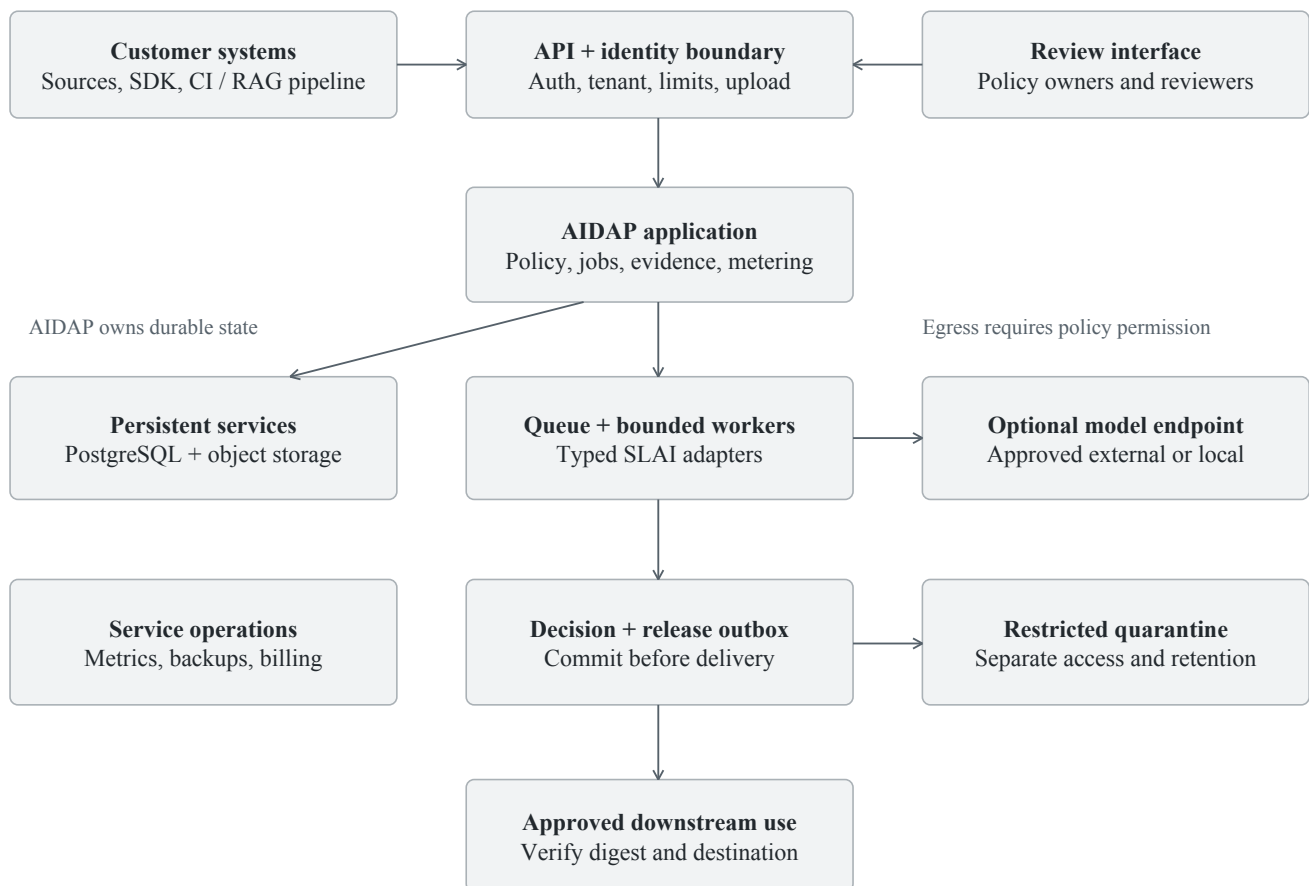


Figure 2. Proposed production architecture and trust boundaries. The SLAI worker is an evaluation component. The application owns identity, policy, persistence, review, metering, and release authorization. External inference is optional and destination-controlled.

The frontend should show the decision, blocking reasons, evaluated coverage, and next action before detailed scores. Reviewers need source spans or redacted examples, policy provenance, and differences from the previous approved version. Dashboard decoration is secondary to resolving an exception quickly and correctly.

PostgreSQL should hold relational state and an append-only event history with constrained service permissions. Object storage should hold larger artifacts and evidence attachments. Store only hashes and pointers in the queue. A cache is optional initially; when introduced, key it by tenant, content digest, normalized representation version, policy, evaluator/model, baseline, destination, and relevant time window. Freshness-dependent results need expiry even when content is unchanged.

Separate worker classes only when workload evidence requires it: CPU checks, document parsing, and optional model evaluation have different resource envelopes. Do not share mutable agent memory across tenants. For stronger isolation, use short-lived processes or tenant-specific worker pools; a Python lock is not a security boundary. Restrict network egress and prohibit evaluation content from selecting arbitrary tools or destinations.

Metering should be generated from durable stage-completion records. Retries must distinguish billable new work from replay of an already committed result. Billing failure must not corrupt assurance results, and assurance failure must not be converted into pass because a payment system is unavailable. Plan exhaustion should hold new optional work predictably and notify the customer.

7.6.2 Deployment choices

Table 7. Deployment responsibilities and trade-offs. “Private” describes a deployment boundary; it does not automatically establish privacy, security, or legal compliance.

Dimension	Shared SaaS	Private cloud / VPC	Self-hosted / on-premises
Operator	AIDAP	Contractually specified provider or customer	Customer, with bounded vendor support
Data plane	Shared service with enforced tenant isolation	Dedicated resources and customer-approved networking	Customer infrastructure; potentially offline
Control plane	Hosted accounts, policy, billing, review	Decide whether metadata may leave VPC; do not assume	Local administration and licensing workflow
Inference	Opt-in approved external provider or local service	Customer-approved endpoint; private model optional	Local or explicitly permitted external endpoint
Advantages	Lowest onboarding and operational cost	Stronger isolation and network control	Maximum operational control and restricted-network fit
Burdens	Tenant security and data-processing obligations	Dedicated upgrades, support, network integration, higher minimum contract	Version fragmentation, customer dependencies, patch delivery, backup responsibility
Initial priority	Primary commercial service after hardening	Paid design partners only	Limited supported distribution after repeatable demand

Start in one selected region with a documented data flow and no external inference by default. Regional hosting must cover storage, backups, logs, support access, and model processing, not merely the API endpoint. Private deployments should use the same application and policy contracts as SaaS, with configuration differences rather than customer-specific code forks.

Use ordinary container deployment and managed services first. Kubernetes, multi-region active-active execution, a distributed event mesh, and a GPU fleet are unjustified before load, recovery, or contractual requirements demand them. Define backup recovery objectives and test restoration before promising an uptime or recovery SLA.

7.7 Target audiences

The best initial customer has recurring input changes, a visible owner of ingestion quality, an integration path that can enforce a hold, and enough engineering cost to make a small subscription worthwhile. Organizations merely experimenting with a chatbot often lack recurring need. Organizations requiring extensive certification, procurement, and custom deployment may have strong need but be uneconomic for a small supplier at launch.

The following buying-cycle and budget assessments are planning hypotheses. They require interviews and paid-pilot evidence; they are not measured market statistics or commitments by the identified segments.

Table 8. Audience assessment and buying hypotheses. Grouping reflects shared workflows; it does not imply uniform budgets across organizations.

Segment	Severity, current workflow, and buying roles	Commercial fit and requirements
AI startups and SaaS vendors adding RAG	Repeated document changes, improvised scripts, manual spot checks. User: AI engineer; buyer: CTO/engineering lead; budget: product engineering.	Best beachhead when production use is recurring. Hypothesis: weeks rather than quarters to buy, €39-499/month for bounded self-service value. Expect API/SDK, CI integration, clear failures, and low latency overhead. Main objection: “we can build this.” Retention depends on ongoing releases and useful review history.
RAG and agent developers / AI consultancies	Multiple customer pipelines with inconsistent intake rules. User: implementation engineer; buyer/budget: consultancy owner or delivery lead.	Strong channel if reusable templates reduce project work. Need separate client projects, exportable evidence, usage caps, and customer-owned credentials. Short pilot cycle; larger rollout follows client approval. Objection: margin erosion and client data access. Retention can be high across repeat projects.
Data engineering and platform teams	Warehouse tests and observability already exist; AI-specific document/context controls are fragmented. User: data engineer; buyer: platform manager; budget: data platform.	Secondary target. Need integration with existing contracts and monitoring, not replacement. Hypothesis: one to three months for a bounded project. WTP rises with cross-team reuse; objection is duplicate tooling. Retention improves when policies attach to stable assets.
Enterprises using sensitive internal knowledge	Manual approval, security review, and inconsistent AI ingestion. User: AI platform team; buyer: platform/AI leader; budget: technology, with CISO and procurement approval.	High severity, longer sales and security cycles. Need SSO, RBAC, deletion evidence, DPA, audit export, regional/private deployment. Annual contracts may be meaningful but support-heavy. Objection: supplier maturity and incumbent consolidation. Strong retention if embedded in release controls.
Local/private AI operators and software vendors embedding assurance	Custom local checks and fragmented evaluator tooling. User: architect; buyer: product or infrastructure head; budget: platform/product.	Attractive later licensing niche. Need offline-capable operation, model independence, packaging, version support, and embedding terms. Months of integration likely. Objection: operational burden and license economics. Retention tied to product releases and support quality.

Segment	Severity, current workflow, and buying roles	Commercial fit and requirements
Professional services firms	Knowledge repositories contain client-confidential material and conflicting versions. User: knowledge/AI team; buyer: operations or technology partner; budget: firm technology.	Moderate-to-high need; often limited engineering capacity. Require simple connectors and review ownership. Sales can take months because confidentiality approval matters. WTP depends on saved review time; low-code expectations can increase support costs.
Financial organizations	Sensitive records, policy evidence, model-risk processes, and substantial governance controls. User: AI/data teams; buyer: platform or model-risk leader; budget: technology with risk/procurement oversight.	Later market. Need auditable controls, domain-specific validation, private deployment, resilience, and vendor diligence. High potential contract value does not imply short cycles; six months or more is a planning assumption. Avoid automated credit or trading fitness claims initially.
Healthcare and life sciences	Sensitive health data, specialized terminology, and high consequences from incorrect context. User: informatics/data team; buyer: technology/research leader; budget: institution or program.	Later, domain-partner-led market. Require validated language/domain coverage, access controls, and jurisdiction-specific review. No claim of clinical suitability. Objection: detector error and procurement burden. Retention may be high after validation but implementation cost is substantial.
Government and public sector	Procurement, records obligations, accessibility, confidentiality, and potentially restricted networks. User: digital/data team; buyer: program/technology authority; budget: public procurement.	Defer direct entry. Expect lengthy tender/security cycles, regional or on-premises operation, and support commitments. Channel partners may be necessary. Budget availability and eligibility cannot be inferred from problem severity.

The recommended beachhead is a software company with an established RAG feature and a recurring knowledge-base update process, using text/metadata inputs and an engineering-owned release pipeline. The initial promise should be measurable: prevent specified ingestion defects and reduce review effort while preserving evidence. A team without a controllable ingestion boundary is a poor pilot because AIDAP cannot demonstrate enforcement.

Qualify prospects by asking for the last three input-related incidents, monthly update volume, existing checks, time spent reviewing exceptions, destination restrictions, and the person authorized to change the ingestion workflow. A willingness to share sanitized examples and pay for a bounded pilot is stronger demand evidence than enthusiasm for the category name.

7.8 User personas

Personas should determine product behavior, not merely marketing copy. Engineers need actionable errors and reproducibility; security teams need data-flow evidence; managers need a credible account of value and responsibility.

Table 9. Personas and measurable outcomes. Benefits are hypotheses to test against a baseline, not demonstrated AIDAP customer results.

Persona	Recurring problem	AIDAP value and measurement
AI engineer	Knowledge updates degrade answers and require manual investigation.	Gate indexing by profile; compare approved versions. Measure integration time, preventable defect escapes, and investigation minutes per incident.
ML engineer	Training batches contain leakage, missingness, or shifted populations.	Reuse structured checks and approved baselines. Measure validated batch coverage and adjudicated defect detection; do not equate drift with model-performance loss.
Data engineer	Schema and metadata changes break downstream jobs.	Enforce explicit contracts with machine-readable findings. Measure failed downstream jobs attributable to known input defects and repair turnaround.
CTO	Too many fragmented checks compete with feature delivery.	Provide a bounded service and transparent costs. Measure engineering hours saved, adoption across projects, and total cost against an internal implementation.
CISO	Sensitive content can reach inappropriate destinations.	Destination-aware screening, restricted processing, and audit evidence. Measure tested leakage scenarios, policy bypass attempts, and deletion completion.
Compliance manager	Evidence of control operation is scattered and difficult to reconstruct.	Export versioned decisions, ownership, exceptions, and retention records. Measure time to assemble a specified control sample, not “compliance percentage.”
AI governance lead	Different teams use inconsistent intake requirements.	Publish approved task profiles and exception rules. Measure policy adoption, unresolved exceptions, and stale policy versions.
Data platform manager	New assurance tooling duplicates existing quality systems.	Consume existing test results and add a release decision layer. Measure duplicated checks avoided and integration maintenance burden.
SaaS founder	Needs trustworthy AI features without enterprise-platform expense.	Predictable subscription, capped model use, and self-service onboarding. Measure activation, customer-reported incidents, and monthly contribution margin.

Review screens should present the minimum evidence required to decide. A CISO should not need to inspect agent internals to understand a blocked destination; an engineer should not need a compliance vocabulary to fix a missing source revision. Role-specific views should share the same underlying decision record.

7.9 Use cases

Table 10. Practical workflows and implementation boundaries. AIDAP should launch with the first, third, sixth, and eighth workflows as a coherent ingestion product, with privacy screening limited to tested coverage.

Use case	Workflow and result	Availability boundary and value
Documents before RAG indexing	Submit text, metadata, source revision, and target collection; validate required fields, freshness, duplicates, permitted source, and privacy; index approved digest only.	Existing checks plus adapters and new product gate. Initial priority: fewer avoidable corpus defects and clearer release ownership.
PII before external LLM use	Screen locally; evaluate destination policy; redact supported entities or block; recheck transformed payload before provider submission.	Privacy implementation exists but detector coverage and destination enforcement need hardening. Reduces specified disclosure paths; does not guarantee detection of all personal data.
Customer-uploaded datasets	Validate CSV/JSON schema, ranges, missingness, duplicates, and size; return row/field-level repair evidence.	Strong fit for existing structural/statistical kernel. Product upload security and bounded parsing are additional work. Reduces onboarding support.
Training-data batches	Check schema, labels, configured leakage heuristics, duplicates, and population changes; require approved baseline and dataset documentation.	Existing quality checks help. General label-correctness estimation or bias certification is not provided. Compare methods such as confident learning only where their assumptions and model probabilities are available (Northcutt et al., 2021).
Drift in AI input data	Profile comparable batches by tenant/source/version; alert or review statistically meaningful changes; owner approves baseline update.	Statistical comparison exists. Product schedules and baseline governance are new. Value is earlier investigation, not proof of performance degradation.
Knowledge-base updates	Diff new and approved revisions; check effective date, owner, source class, and supersession; release atomic collection update.	Metadata checks are near-term; collection-level atomic delivery is integration work. Prevents accidental coexistence of obsolete policy versions.
Contradictory documents	Identify candidate pairs by topic/revision; compare scoped claims; show cited spans; quarantine or review unresolved conflicts.	General cross-document semantic contradiction is substantial new development. Avoid all-pairs cost and false conflicts between different dates/jurisdictions.
Required metadata	Enforce owner, source, effective date, classification, permitted use, and revision before ingestion.	Strong deterministic MVP use. Makes provenance requirements operational and repairable.
Third-party API feeds	Validate envelopes and domain values; compare volume/freshness; bind findings to supplier and feed version.	Existing checks plus supported connector. Network credentials, rate limits, and supplier-change management remain product work.
Retrieved-context sufficiency	Evaluate the query, task rubric, and selected passages; identify missing evidence; instruct application to retrieve again or abstain.	Substantial new evaluator and integration work. Research evaluation frameworks are useful foundations, not proof of domain adequacy (Es et al., 2024).
Organization-specific policies	Combine approved source classes, destinations, retention, mandatory checks, and review roles in versioned profiles.	New AIDAP policy layer around existing findings. Creates recurring control value without requiring every rule to use a model.

Use case	Workflow and result	Availability boundary and value
Agent memory before reuse	Check memory origin, age, permissions, and applicable task; exclude expired or untrusted entries; preserve decision links.	Metadata policy reuse is plausible; memory semantics and agent-framework adapters are new. Limit contamination and unauthorized reuse; do not claim complete agent safety.

For each pilot, choose one release boundary and a small set of preventable defects. Run in shadow mode first to collect findings without interrupting production; adjudicate a representative sample of both flagged and unflagged inputs. Enforce only after the customer approves thresholds and ownership. Shadow mode must be visibly distinguished from an active gate.

7.10 Competitive market analysis

7.10.1 Market structure and evidence conventions

The relevant market spans data quality and observability, AI evaluation and observability, inline guardrails, and do-it-yourself engineering. Product categories increasingly overlap. Public pages establish advertised features and list prices; they do not establish comparative accuracy, negotiated enterprise costs, or the absence of an unadvertised capability. “Gap” below is an assessment of fit with AIDAP’s proposed narrow workflow, not proof that a competitor cannot implement it.

The customer can often combine existing tools at low software cost. AIDAP must therefore compete with the engineering and maintenance cost of that combination, not only with high-priced enterprise platforms. Open-source checks, a queue, a database, and an LLM evaluator form a credible substitute. An integrated evidence and release workflow must justify the subscription.

Table 11. Competitive landscape as of 17 September 2026. USD list prices are not normalized comparisons: traces, spans, processing units, features, model charges, retention, and contract terms differ. “No numeric list price observed” does not mean that the vendor has no published pricing elsewhere.

Vendor	Expertise, users, and advertised services	Deployment and published pricing	Fit, strengths, limitations, and threat
Monte Carlo	Enterprise data/AI platform teams; current positioning includes Agent Trust and data observability.	Enterprise service; deployment/security terms require confirmation. Pricing by request.	Strength: broad operational context and enterprise relationships. AIDAP fit gap: a small, self-service intake workflow must be demonstrated, not assumed absent. Threat: high as AI trust expands. (Monte Carlo, n.d.-a, n.d.-b)
Soda	Data engineers and platform teams; quality checks, contracts, and collaborative quality workflows.	Free tier; Team from \$750/month with Soda Processing Units and usage terms; Enterprise custom, including private options.	Strength: pipeline-oriented quality and transparent entry anchor. Less specifically centered on task-conditioned AI intake in the cited offer. High threat for structured checks; potential integration partner. (Soda, n.d.)
Anomalo	Enterprise data teams; automated quality/anomaly detection and advertised unstructured-document monitoring.	SaaS or customer VPC options advertised; no numeric list price observed.	Strength: automation and direct expansion into document quality. Do not characterize as tabular-only. AIDAP must compete on workflow economics and task profiles. High threat. (Anomalo, n.d.)

Vendor	Expertise, users, and advertised services	Deployment and published pricing	Fit, strengths, limitations, and threat
Bigeye	Data/platform teams; data observability and AI Trust positioning.	Enterprise-led offer; exact private-deployment and price terms require vendor confirmation.	Strength: established data-monitoring focus. Evidence-reviewed pricing gives no small-team cost benchmark. AIDAP gap is a proposed compact intake product, not a verified feature absence. Medium/high threat. (Bigeye, n.d.)
Metaplane / Datadog	Data teams; monitoring, lineage, and development workflow integrations; acquired by Datadog in 2025.	Cloud-oriented service and warehouse integrations; current commercial terms should be confirmed.	Strength: distribution and broader observability adjacency. AIDAP should integrate rather than replace warehouse monitoring. High bundling threat for shared customers. (Metaplane, n.d.; Datadog, 2025)
Acceldata	Enterprise data-platform operations; data reliability, profiling, reconciliation, lineage, and pipeline/compute oversight.	Hybrid enterprise positioning; trial/demo, no numeric list price observed.	Strength: broad operational coverage. Breadth can exceed a small intake-gate requirement, but procurement fit is customer-specific. Medium threat initially, higher in enterprise expansion. (Acceldata, n.d.)
Great Expectations	Developers and data teams; expectations-based testing and managed collaborative workflows.	Open-source GX Core plus cloud Free Developer, Team, and Enterprise offers; no numeric paid rate visible on the reviewed pricing page.	Strength: programmable deterministic checks and DIY credibility. AIDAP must add release, review, and evidence value beyond rules. High substitute threat; useful interoperability target. (Great Expectations, n.d.)
Arize / Phoenix	AI/ML teams; tracing, evaluation, experiments, and production analysis.	Phoenix open-source/local option; Arize Free, Pro \$50/month, Enterprise custom with SaaS/self-hosted options advertised. Pro includes 50,000 spans and 10 GB under published terms.	Strength: accessible developer ecosystem and AI lifecycle visibility. AIDAP's specific asset-release workflow remains a positioning choice, not exclusive capability. High adjacent threat. (Arize, n.d.)
Galileo	AI teams; evaluation, observability, and real-time guardrails, with enterprise private deployment.	Free 5,000 traces; Pro \$100/month billed yearly with 50,000 traces; Enterprise custom, including VPC/on-premises options.	Strength: broad AI evaluation plus preventive controls. Local/private operation and guardrails are not unique AIDAP differentiators. High direct/adjacent threat. Corporate Cisco-related developments should be confirmed during procurement. (Galileo, n.d.)

Vendor	Expertise, users, and advertised services	Deployment and published pricing	Fit, strengths, limitations, and threat
Giskard	AI builders and risk teams; adversarial testing, RAG checks, red teaming, and review workflows.	Free local open-source offering; Enterprise quote; SaaS, private cloud, and on-premises options advertised.	Strength: testing depth and adversarial focus. AIDAP would emphasize operational intake decisions, while Giskard remains a strong evaluation substitute/partner. High threat for safety-led positioning. (Giskard, n.d.)
Patronus AI	Current site emphasizes frontier research, simulation, and Digital World Models; evaluation research includes Lynx, GLIDER, and FinanceBench.	Current availability, deployment, and pricing of earlier evaluation-service offers require confirmation.	Strength: evaluator research and domain benchmarks. Do not assume historical SaaS packaging remains unchanged. Medium adjacent methodological threat; uncertain direct commercial overlap. (Patronus AI, n.d.)
Cleanlab / Handshake	Data-quality and label/reliability expertise; prior commercial positioning addressed hallucinations and knowledge gaps. Handshake acquisition announced January 2026, with focus on frontier training-data research.	Open-source continuation stated; availability and pricing of prior standalone commercial offers require confirmation.	Strength: data-centric methodology. Historical SaaS comparisons can mislead after acquisition. High methodological relevance; current direct buying alternative uncertain. (Cleanlab, 2026)
Fiddler	Enterprise AI control plane; inline request/response screening and AI monitoring, including advertised PII/PHI/secrets controls.	Free guardrails; Developer \$0.002/trace; Enterprise quote, with SaaS/VPC/on-premises options advertised.	Strength: direct pre-inference overlap and enterprise deployment. AIDAP cannot claim preventive screening as an empty market. High direct threat; compete on narrow intended-use workflow and measured economics. (Fiddler, n.d.-a, n.d.-b)
Langfuse	Developers; open-source LLM engineering, tracing, and evaluation.	Self-hosting; hosted Hobby free and Core from \$29/month plus usage.	Strength: low entry cost and developer adoption path. AIDAP must deliver materially different intake enforcement, not a second tracing dashboard. High DIY/adjacent substitute threat. (Langfuse, n.d.)

7.10.2 Commercial implications

The market is crowded around quality checks, traces, evaluation, and guardrails. AIDAP should not infer a large addressable market merely by adding these categories together; their budgets overlap. No unsupported TAM estimate is necessary to make the initial business case. A bottom-up plan based on reachable accounts, activation, retained subscription value, and support capacity is more useful.

Low public AI-tool entry prices create a strong anchor. A €149 monthly offer must explain why intake control and evidence are worth buying in addition to tools a customer already uses. Enterprise observability prices do not validate a small customer's willingness to pay. Conversely, a low monthly price cannot fund repeated bespoke security reviews or private installations.

The most useful competitive evaluation is a controlled customer trial. Compare AIDAP, the customer's existing stack, and a simple scripted baseline on the same defect set. Measure precision, recall, review burden, integration effort, release integrity, and total delivery cost. Competitor claims of accuracy, latency, or savings should not be repeated as independent evidence without equivalent test conditions.

7.11 Competitive positioning

Recommended category: **AI Data Assurance**, with **AI Input Assurance** as the operational description and **data fitness for AI** as the customer problem. The positioning statement is: “AIDAP validates data and context against the requirements of a specific AI use, then provides an enforceable release decision and the evidence behind it.”

For the beachhead, a more concrete statement is: “AIDAP gives RAG teams a controlled intake gate for knowledge updates, combining tested data checks, destination-aware privacy policies, review, and versioned release evidence.” This is narrower, more testable, and easier to integrate than a promise of universally trustworthy AI.

Table 12. Deliberate competitive boundaries. These are strategic recommendations, not limitations imposed on customer integrations.

Competing as a generic...	Why this is strategically weak	Preferred boundary
Data observability platform	Requires broad connectors, lineage, warehouse expertise, and enterprise sales against established vendors.	Consume monitoring signals; focus on admission decisions for AI use.
LLM evaluation platform	Low-cost and open-source alternatives already cover traces, experiments, and judge workflows.	Use evaluators as components of a task-specific gate.
AI monitoring platform	Implies broad runtime coverage, dashboards, and incident operations beyond the initial kernel.	Export decisions to existing observability tools.
Warehouse tool	Structured-data scanning alone does not justify AI-specific positioning.	Support structured inputs when connected to a declared downstream AI task.
Generic guardrail platform	Direct competition includes mature privacy/safety vendors and provider-native controls.	Combine bounded screening with source fitness, review, and evidence.

The differentiating bundle should include task profiles, deterministic-first evaluation, explicit unknown states, configurable release policies, reproducible evidence, and affordable operation in the customer's permitted environment. Each element is replicable. The advantage must be demonstrated in the quality of the assembled workflow, not asserted from the feature list.

Claims to avoid include “eliminates hallucinations,” “GDPR compliant by default,” “understands all documents,” and “unique multi-agent assurance.” Claims to earn include measured performance on named defect classes, documented handling of unsupported inputs, tested recovery, transparent cost caps, and successful integration within a specified customer workflow.

7.12 Why SLAI instead of a simple LLM API wrapper

An LLM API is useful for flexible interpretation, classification, and bounded semantic judgments. It is not by itself a durable policy system, a deterministic schema validator, or a reliable release mechanism. SLAI's value is reusable local validation and structured component organization; the commercial product must still implement operational control around it.

Table 13. Architectural comparison. A well-engineered API-based system can implement many of the same controls. SLAI reuse is advantageous only where it saves validated engineering effort without importing disproportionate complexity.

Dimension	SLAI-based AIDAP approach	Mainly model-API approach
Specialization	Existing quality/privacy/safety components can return separate findings.	Separate prompts or tools can simulate specialization; architecture must be built. Multiple calls do not guarantee independent evidence.
Determinism	Local schema/range/requiredness checks can be reproducible.	Asking a model to check simple constraints adds cost and variability; ordinary code can and should be added.
Semantic breadth	Current SLAI semantic rules are bounded.	Strong external models may provide broader language understanding, with measured limitations.
Local execution/privacy	Many checks need no external provider.	External calls require approved data handling; local models are an alternative with operational cost.
Model independence	AIDAP-owned adapters can keep policy/evidence stable across models.	Provider-specific APIs create switching work; clean adapters can reduce it here too.
Checkpoints and recovery	Existing framework machinery provides a starting point, requiring validation.	Must be implemented in application orchestration; API completion alone provides no durable workflow.
Quality gates/audit	SLAI reports can feed a deterministic AIDAP gate.	Entire gate/evidence layer must still be built; model rationale is not an audit trail.
Training/tuning	Offline training/tuning components exist, but useful trained task models remain to be developed.	Hosted providers reduce training operations; customization may be constrained or costly.
Costs and availability	Cheap local rules avoid unnecessary inference; dependencies and maintenance remain real costs.	Fast prototype, variable inference spend, provider limits, and network dependency.
Security/defensibility	Narrow framework reuse can improve consistency; unnecessary components increase attack surface.	Smaller prototype surface, but prompt injection and untyped judgments remain risks. Neither architecture alone is a moat.

External models remain attractive for nuanced language, long-tail terminology, and early task-specific prototypes. AIDAP should route only necessary, permitted content to them after local checks, use structured outputs, and require evidence spans where possible. LLM judges can exhibit position, verbosity, and self-preference biases; ensemble agreement is not independence when evaluators share models or prompts (Zheng et al., 2023).

The build decision should be revisited empirically. If a SLAI component's dependency, hardening, and maintenance costs exceed those of a small established library for the same tested function, AIDAP should keep its contract and substitute the implementation. Commercial defensibility must reside in reliable policies and workflows, not an obligation to use every framework component.

7.13 Business model

7.13.1 Pricing architecture

The recommended model is a predictable subscription with included deterministic capacity, a small clearly identified managed-inference budget, and optional prepaid extensions. Disable automatic overage by default. Customers should be able to choose “hold when budget exhausted,” receive advance alerts, and see a cost estimate before a large run. BYO model credentials reduce provider pass-through but do not eliminate AIDAP compute, support, or metering cost.

Table 14. Business-model alternatives. Recommended packaging balances customer predictability with enforceable resource limits.

Model	Assessment for AIDAP
Freemium / free developer	Useful for activation and integration testing if bounded by capacity, retention, and support. Unlimited free inference is economically unsafe.
Monthly subscription	Best low-friction entry; predictable bill and easy cancellation. Requires self-service onboarding.
Annual subscription	Useful after retention is demonstrated; modest discount should reflect cash-flow/support value, not conceal weak adoption.
Pure usage pricing	Aligns with cost but creates bill anxiety and volatile revenue. Prefer capped usage within a subscription.
Per source / per project	Easy to understand but weakly correlated with workload. Use project count as packaging, not the primary cost meter.
Per validation run	Simple but unfair across tiny and huge inputs. Pair with explicit size/compute bounds and idempotency.
Per million rows	Useful for homogeneous tabular batches; poor for documents and wide records. Offer only as an estimate or specialized pack.
Per GB	Useful deterministic scan meter if decoded bytes, minimums, and repeated scans are defined. Does not cover semantic complexity.
Enterprise/private/on-premises license	Annual minimum plus scoped onboarding and support. Customer infrastructure cost is separate; price must cover version and deployment obligations.

7.13.2 Proposed pricing experiment

Table 15. AIDAP pricing hypotheses, not validated prices or current offers. All capacity figures are proposed maximum included envelopes subject to benchmarked per-run limits; they are not measured SLAI throughput commitments. Basic tenant security, deletion, and privacy controls must exist on every hosted tier.

Tier	Monthly price hypothesis	Included envelope and commercial purpose
Developer / Community	€0	Local SDK/examples and hosted sandbox: 100 runs, 0.1 GB scan allowance, seven-day evidence retention, no managed-model budget, community support. Synthetic/non-sensitive evaluation first.
Starter	€39	One project; 1,000 runs and 5 GB deterministic scans; €3 managed-inference allowance; 30-day evidence; self-service support. A single production ingestion path.

Tier	Monthly price hypothesis	Included envelope and commercial purpose
Team	€149	Five projects; 10,000 runs and 50 GB scans; €15 managed-inference allowance; 90-day evidence; review roles, schedules, and webhooks. Shared operational workflow.
Business	€499	Twenty projects; 50,000 runs and 250 GB scans; €50 managed-inference allowance; one-year evidence, configurable retention, audit export, and priority business-hours support. Publish entitlements only when implemented.
Enterprise	From €12,000-30,000/year, subject to scope	Private deployment, supported SSO integration, contractual support, and security review; separate €3,000-8,000 onboarding hypothesis. Infrastructure and unusual integrations priced separately.

Both run and scan limits apply. Define one run as one submitted asset version and policy evaluation, with a published maximum decoded size and bounded standard checks. A deterministic scan GB is 10^9 decoded bytes examined; document expansion, repeated scans, and minimum billable run size must be explicit. Complex joins, all-pairs comparisons, OCR, and custom compute are excluded initially. Record internal CPU-seconds and memory to detect whether the public meter adequately predicts cost.

Managed inference should use a visible prepaid euro balance with published conversion/routing terms, not opaque “AI credits.” The provider-rate example in Section 7.14 is in USD; AIDAP must manage conversion and procurement separately. BYOK customers still consume orchestration capacity. Optional top-ups should be priced only after benchmarked costs are known; no forced overage is preferable to an unsupported low rate.

Public alternatives in Section 7.10 validate competitive price pressure, not demand for these tiers. Test €39 versus a modestly higher entry price with otherwise comparable cohorts, and test whether Team buyers value shared review and retained evidence. Do not gate essential security behind Enterprise. Offer an annual option only after observing repeat use; a 10-15% discount is a hypothesis to evaluate against cash flow and churn.

7.13.3 Willingness-to-pay validation

Recruit ten to fifteen qualified interviews and three to five paid pilots as initial operating targets. Ask buyers to compare the proposed subscription with their own engineering/review cost and existing tool spend. A four-week pilot priced at €500-1,500 can include one integration and a defined defect set; this is a proposed services price, not recurring revenue. Set explicit limits so the pilot does not become unpriced consulting.

A successful pilot should establish recurring input volume, an owner of exceptions, measurable time saved or defects intercepted, acceptable false-positive burden, and willingness to continue at a stated subscription. If customers only want one-time dataset cleanup, the recurring-service thesis is weak. If they need extensive custom connectors before value appears, narrow the segment or raise the contract minimum.

7.14 Cost model and unit economics

7.14.1 Direct costs and operating expenses

Direct delivery costs include worker compute, API/database allocation, object storage, backup storage, networking, inference and third-party APIs, service observability, transactional email, payment fees, and direct customer support. Development, general administration, sales, and much security/compliance program work are operating expenses. Accounting treatment should be consistent; excluding founder support from unit economics understates the cost to serve.

Use the following model: **monthly contribution = recognized subscription revenue – direct infrastructure – managed inference – payment costs – allocated delivery support**. Gross margin is this contribution divided by revenue under the stated cost classification. Company break-even additionally requires covering fixed operating expenses and founder/employee compensation.

Table 16. Illustrative monthly unit economics. Every cost is an assumption, not a supplier quote or measured AIDAP result. The compute allocations represent a normal-use cohort, not demonstrated costs at full plan limits. Taxes and fixed operating expenses are excluded.

Illustrative cohort economics	Starter	Team	Business
Monthly revenue	€39	€149	€499
Direct platform/compute/storage allocation	€5	€12	€35
Managed-inference consumption assumption	€3	€15	€50
Payment assumption: 3% + €0.30	€1.47	€4.77	€15.27
Support at €40/hour	€4 (0.1 h)	€10 (0.25 h)	€40 (1 h)
Total direct cost	€13.47	€41.77	€140.27
Contribution / gross margin	€25.53 / 65.5%	€107.23 / 72.0%	€358.73 / 71.9%

The model is most sensitive to support and unbounded semantic work. One additional support hour reduces Starter contribution by €40, making it negative. Doubling Business inference consumption from €50 to €100 reduces contribution to €308.73 and margin to 61.9%. This is why allowances, self-service repair guidance, and private-deployment minimums matter more than a nominally cheap cloud server.

At an illustrative €8,000 monthly fixed operating cost, a homogeneous Team cohort contributing €107.23 each requires approximately 75 active accounts to cover that cost. At €15,000, approximately 140 are required. These are arithmetic scenarios, not acquisition forecasts; mixed tiers, churn, annual discounts, failed payments, and founder capacity change the result. Early infrastructure underutilization can also make initial margins worse than cohort allocations suggest.

7.14.2 Inference sensitivity

Anthropic's published pricing lists Claude Haiku 4.5 at \$1 per million input tokens and \$5 per million output tokens, and Sonnet 4.6 at \$3 and \$15 respectively; batch pricing can reduce eligible rates (Anthropic, n.d.). For 10,000 documents with 4,000 input tokens and 300 output tokens per evaluation, one Haiku-rate pass costs **\$55** before retries or other charges: 40 million input tokens plus 3 million output tokens. Three passes cost \$165. At the cited Sonnet rates, the corresponding totals are \$165 and \$495.

These examples are provider-cost arithmetic, not a recommendation that either model meets AIDAP's accuracy needs. Long documents, pairwise comparisons, retries, and verbose rationales can dominate cost. If only 10% of

documents require the model pass, the first example falls to \$5.50, assuming the same token mix. Selective routing is therefore economically meaningful, but must not omit a mandatory check merely to meet a budget.

Cache only decisions whose full context remains valid. Reuse approved baselines and deterministic fingerprints; batch offline work; limit output schemas; route low-risk tasks to adequate cheaper models; and expose evaluation depth. Local/open models can avoid per-call provider charges but incur memory/GPU utilization, deployment, patching, and evaluation costs. For low volume, an idle local inference server can be more expensive than carefully capped APIs.

7.14.3 Cost controls and measurement

Meter CPU time, peak memory, decoded bytes, tokens, retries, artifact retention, and support by tenant and workload. Benchmark plan-limit scenarios before publication. Set per-run limits, concurrency quotas, stage timeouts, and daily spend caps. Alert on rising cost per accepted asset and repeated customer misconfiguration. Reserve model calls for questions that deterministic rules cannot answer adequately.

Security and compliance costs must be planned even when no certification is pursued. Dependency maintenance, incident response, vulnerability remediation, access reviews, restore tests, and customer security questionnaires consume scarce company capacity. Buy commodity billing and identity services where practical, but review their data flows and terms. VAT/tax handling should be automated through a suitable billing/tax provider with professional accounting review.

7.15 Scalability strategy

Scaling should follow measured bottlenecks. More organizations do not automatically require a more distributed architecture; workload mix, artifact size, latency requirements, and isolation obligations matter more than account count.

Table 17. Staged scaling plan. Each change should resolve an observed constraint and retain a manageable operating model for a small company.

Operating stage	Architecture and operating focus	Trigger for the next change
First users	One API service, managed PostgreSQL, object storage, queue, and small worker pool. Manual onboarding; bounded formats and rules.	Demonstrated recurring use, known failure modes, and stable per-run costs.
Tens of organizations	Horizontal workers, per-tenant quotas, durable usage ledger, reliable retries, support runbooks, and deletion/restore tests.	Queue delay, noisy-neighbor impact, or support load exceeds targets.
Hundreds	Separate CPU/model queues, autoscaling, tenant-aware scheduling, database index tuning, and time-based audit retention.	Sustained query/storage pressure or contractual isolation needs.
Thousands	Partition high-volume event/usage tables where measured; dedicated pools for large tenants; regional cells only when demand supports them.	Demonstrated regional, throughput, or blast-radius requirement - not account count alone.

Keep API services stateless and move expensive work to bounded workers. Use queue visibility leases, retry limits, dead-letter queues, and explicit cancellation. Prioritize paid interactive work without starving batch jobs. A single customer's large corpus must not monopolize workers or model-provider rate limits. Reserve capacity and retry budgets by tenant and provider.

PostgreSQL should remain the source of truth for job and decision state. Partitioning is useful for large append-only event tables after indexing and retention are insufficient; premature sharding complicates transactions and support. Object storage should hold large evidence, while metadata queries remain relational. Checkpoint retention should be separate from customer evidence retention because recovery state may be short-lived and sensitive.

Model-provider limits need circuit breakers, backoff, and policy-aware fallback. A cheaper or available model may not be an approved substitute. When a mandatory semantic evaluator is unavailable, hold or return an explicit unavailable decision; do not silently lower evaluation depth. Asynchronous ingestion tolerates this behavior more naturally than an inline inference gateway, reinforcing the initial product choice.

7.16 Market-entry strategy

Begin developer-first within engineering-led SMEs, rather than attempting broad enterprise sales or serving hobbyists as the main revenue base. The first sale should attach to a recurring release problem with a measurable owner and budget. An API-first experience and an open-source SDK reduce integration risk; a small review console completes the workflow for non-engineering stakeholders.

Publish one excellent integration guide: validate a knowledge update, inspect findings, repair it, and index only the approved artifact. Provide synthetic examples of stale metadata, duplicate records, a restricted identifier, and an unsupported input. A working failure example is more persuasive than a large catalog of planned integrations. GitHub examples and technical content should explain limits and measurements, not imply universal assurance.

Use AI engineering communities and direct outreach to teams already operating RAG pipelines. Partner with two or three consultancies only after the generic API is stable. Partners should use reusable profiles and earn referral or implementation revenue; they should not require separate product branches. Cloud marketplaces and procurement frameworks belong later, when annual contracts justify their operational overhead.

Table 18. Initial go-to-market sequence. Time windows are planning estimates contingent on engineering capacity and pilot access.

Period	Commercial activity	Decision gate
Months 1-3	Interview qualified teams; obtain representative sanitized failure examples; recruit paid design partners.	At least one repeated problem across several prospects and a feasible enforced integration.
Months 4-6	Run scoped pilots; compare with existing scripts; test price and review burden.	Customers continue paying for recurring use; no unresolved release-integrity issue.
Months 7-9	Productize onboarding, billing, one connector, and reference evidence; reduce support.	Activation and contribution margin support self-service growth.
Months 10-12	Expand the repeatable segment and partner channel; selectively assess private deployments.	Retention and support capacity justify broader sales, rather than growth in unpaid experiments alone.

The company should decline prospects whose essential requirement is a full governance suite, a general data catalog, or a bespoke enterprise connector before first value. Such opportunities can generate revenue while preventing a repeatable product from emerging. A small company needs a clear support boundary and a credible path to serving the next customer with substantially the same software.

7.17 Keeping AIDAP ahead of competitors

Defensibility should be assessed by replacement difficulty and demonstrated performance, not novelty of terminology. A multi-agent implementation, prompt collection, dashboard, or generic PII rule list is easy to copy. Local deployment is valuable to some buyers but already available from several alternatives. An open framework license or model-independent API is not itself a proprietary moat.

Table 19. Defensibility assessment. Customer lock-in should arise from useful accumulated workflow value, not obstructed export or proprietary access to customer data.

Potential advantage	Strength and conditions
Task-specific fitness profiles	Moderate initially; stronger when validated against real failure distributions and maintained as policies change.
Customer policies, contracts, and historical baselines	Meaningful retention value when integrated into recurring work. Exportability and customer ownership should remain clear.
Approved/rejected examples and failure taxonomy	Valuable only with lawful rights, consistent labeling, representative coverage, and measurable improvement. Cross-customer reuse cannot be assumed.
Proprietary evaluation datasets	Potentially strong if difficult to reproduce and demonstrably predictive of customer outcomes. Small synthetic collections are weak evidence.
Evidence/provenance relationships	Useful accumulated operational context. A graph database is unnecessary until relationship queries justify it.
Integrations and reliable release enforcement	Strong practical switching cost when maintained well; expensive to build indiscriminately.
Local/private deployment	Differentiating for some segments, but not unique; support quality and upgrade reliability matter.
SLAI evaluator ensembles	A research opportunity, not an established moat. Must outperform simpler baselines after cost, correlated errors, and maintenance are included.

The staged innovation strategy is to improve one task profile at a time. First establish metadata/privacy/quality intake for RAG updates. Next build a benchmark of real, permissioned defects and compare candidate semantic validators. Then add source-supersession and scoped contradiction checks. Only after these demonstrate retained value should AIDAP expand to retrieval sufficiency or agent-memory assurance.

Track incumbent changes quarterly, especially low-cost guardrails, document-quality offerings, and enterprise bundles. If a competitor makes a generic check effectively free, AIDAP should integrate or retire that component rather than defend it as a differentiator. Preserve the value of task policy, evidence, and release integrity while substituting commodity detection technology.

7.18 Product roadmap

Table 20. Phased product roadmap. Phases are gated by evidence; they are not declarations that the capabilities already exist or a commitment to complete all four within twelve months.

Phase	Essential capabilities	Exclusions and dependencies	Commercial objective / difficulty
1 - MVP: core fitness gate	Text/metadata and CSV/JSON inputs; selected SLAI checks; explicit decisions; immutable input binding; minimal evidence; CLI/API; manual but attributable review.	No broad semantic truth claims, autonomous repair, OCR, or connector catalog. Depends on source/version reconciliation, typed adapter contracts, and hardening of failure paths.	Prove one recurring paid intake use case. Medium/high difficulty because release integrity matters more than interface breadth.
2 - Production SaaS	Accounts, tenant isolation, billing/caps, schedules, dashboard, signed webhooks, one supported integration, deletion/backups, operational monitoring.	No arbitrary private deployments or 24/7 bespoke support. Depends on repeatable pilot value, tested isolation/recovery, and manageable direct costs.	Turn pilots into retained subscriptions with low support burden. High product/operations effort outside SLAI.
3 - AI-specific intelligence	Task-specific RAG fitness, scoped semantic consistency, improved privacy/safety coverage, calibrated uncertainty, labeled regression datasets.	No universal fact verification or automatic legal judgment. Depends on domain examples, evaluation methodology, and model-cost controls.	Increase value and expansion revenue through measured detector improvements. High research and validation difficulty.
4 - Platform expansion	Agent-memory/selected agent-input assurance, enterprise policy distribution, governance evidence integrations, supported private deployment.	No full GRC replacement, generic agent runtime, or unrelated monitoring platform. Depends on retention, support capacity, and mature update/security processes.	Larger contracts without losing the intake-control focus. High integration and commercial complexity.

For a small independent company, assume an initial capacity of one experienced full-time builder with limited specialist support, or a similarly constrained team. Under that assumption, twelve months should prioritize a narrow MVP and reliable SaaS operation, with only selected Phase 3 experiments. Adding regulated verticals, private installations, and advanced semantic evaluation simultaneously is not a credible default plan.

Release gates should be explicit: no malformed or missing mandatory result can pass; policy and artifact versions must be bound; cross-tenant access tests must pass; retries cannot duplicate delivery; deletion and restore procedures must be exercised; and paid pilots must demonstrate tolerable review burden. Commercial launch should wait for these controls, even if the interface appears complete.

7.19 Security and privacy

The platform itself processes potentially sensitive information and becomes part of the attack surface it is intended to reduce. Security must therefore be designed around data flows and privileges. Minimize raw-content retention, encrypt transport and storage, separate production access from development, rotate credentials, and restrict worker egress. Store customer API keys in a dedicated secret mechanism; never place them in logs, evidence bundles, or agent shared memory.

GDPR principles include lawfulness, purpose limitation, data minimization, accuracy, storage limitation, and security; personal data can identify someone directly or indirectly (EDPB, n.d.-a). AIDAP should help customers implement selected technical controls, while preserving the distinction between detection and lawful processing. A regex match is not a complete assessment of personal data, and the absence of a match does not establish anonymity.

Table 21. Security and privacy control design. These are requirements for the proposed product; existing SLAI components do not establish their deployment or effectiveness.

Control area	Proposed product requirement
Access control	Tenant-bound authorization on every API/object path; project roles; least-privilege service accounts; MFA for privileged administration.
Data minimization	Process only required fields; redact evidence examples; avoid raw payloads in traces and alerts; configurable short retention.
Retention/deletion	Track source objects, derivatives, evidence, caches, checkpoints, and backups; define deletion SLA and backup expiry; distinguish legal holds where applicable.
Encryption and keys	TLS in transit; managed encryption at rest; documented key access/rotation; customer-managed keys only when supportable.
Model providers	Explicit opt-in per destination and data class; documented subprocessors, regions, retention, and provider terms. BYOK does not remove data-processing responsibilities.
Local/private operation	Verify network egress and telemetry; document residual metadata flows; customer-controlled models need the same evaluation and patch discipline.
Audit and incident response	Record privileged access and decision changes; protect audit integrity; maintain incident triage, notification, and recovery procedures.
Secure development	Dependency inventory, vulnerability scanning, signed releases where supported, secret scanning, least-privilege CI, and patch ownership.

Controller/processor roles, lawful basis, transfers, subprocessors, and data-processing agreements require explicit contractual analysis for the actual service. AIDAP should supply a data-flow description and a security schedule that match implementation. Customers need deletion/export procedures and enough information to assess their own obligations. Sensitive health or other special-category information requires additional care; it should not be accepted into an unqualified launch tier merely because a detector recognizes it.

Security measures should be proportionate to risk and regularly reviewed, including recovery and access controls (EDPB, n.d.-b). Regional hosting is one control, not a complete transfer analysis. Logs, support access, backup replication, and external inference can cross the intended boundary. Privacy testing should verify these paths, not just the text detector.

7.20 EU regulatory environment

7.20.1 AI Act and governance implications

The European Commission's current timeline states that the AI Act generally became applicable on 2 August 2026, with earlier application of prohibited-practice/AI-literacy provisions and GPAI obligations. Its updated page reports the AI Omnibus entering into force on 27 July 2026 and high-risk dates of **2 December 2027** for specified Annex III uses and **2 August 2028** for regulated-product systems. These dates should be checked against the applicable consolidated legislation for a particular deployment (European Commission, n.d.-a).

AIDAP's relevance is evidentiary and operational: documented input requirements, traceable decisions, human review, and reproducible records may support customers' risk-management processes. They do not establish conformity. Classification depends on intended purpose and actual role, not the phrase "assurance platform." AIDAP could also use AI components itself, requiring assessment of its own obligations. The Commission describes high-risk requirements concerning data quality, records, oversight, robustness, and risk management; AIDAP addresses only selected supporting controls (European Commission, n.d.-a).

The NIST AI Risk Management Framework provides a useful voluntary structure for governing, mapping, measuring, and managing AI risks, without constituting an EU compliance certificate (NIST, 2023). AIDAP can align evidence fields and operating reviews to such frameworks while avoiding claims that a framework mapping proves effective risk reduction.

7.20.2 GDPR, cybersecurity, and contracts

GDPR obligations apply according to actual personal-data processing, independently of whether an AI system is categorized as high-risk. Product priorities should include minimization, destination restrictions, retention, access, and evidence of control operation. Legal analysis must address the actual controller/processor relationship and processing purpose; the software cannot infer a complete lawful basis from a consent flag (EDPB, n.d.-a).

The Cyber Resilience Act concerns products with digital elements. The Commission states that reporting obligations apply from **11 September 2026**, with main obligations from **11 December 2027** (European Commission, n.d.-b). AIDAP should assess the scope implications of commercial software distribution and supporting remote services with qualified counsel; it should not assume every SaaS configuration is treated identically. Vulnerability intake, remediation, dependency records, and supported-lifecycle commitments are prudent operating priorities regardless of the final scope conclusion.

NIS2 may matter through entity classification and customer supply-chain/security requirements; applicability should be assessed against the relevant national implementation and service role (European Commission, n.d.-c). Enterprise contracts can additionally impose incident notification, audit, business continuity, subcontractor, and data-location obligations. A small provider should not sign obligations it cannot operationally fulfill.

Maintain a regulatory register with an owner, review date, applicable product/deployment, and action. Treat policy packs as configurable implementation aids, not legal advice. Avoid automatic marketing statements such as "AI Act compliant" or "GDPR certified" in the absence of a precise, valid basis and scope.

7.21 Reliability and validation strategy

7.21.1 Test the gate and the detectors separately

The gate's integrity is primarily a software correctness problem: an unauthorized or incomplete result must not release content. Detector quality is an empirical problem: a check can execute correctly and still miss a defect. Both need evidence. Good detector accuracy cannot compensate for a bypassable release path, and perfect workflow code cannot make an inaccurate semantic evaluator trustworthy.

Table 22. AIDAP assurance strategy. Tests must exercise real failure paths, not merely mirror implementation or validate attractive examples.

Validation layer	Required evidence and release criterion
Unit and contract tests	Schema/type/range checks; policy precedence; malformed results; unknown states; deterministic versioned fixtures. Mandatory failure cannot become pass.
Integration tests	Real adapter execution, database/object/queue boundaries, authentication, tenant scopes, and downstream digest verification.
Golden datasets	Adjudicated valid/invalid examples by defect, language, format, and intended use; immutable versions and separate development/test splits.
Adversarial tests	Prompt injection in documents, obfuscated identifiers, conflicting metadata, oversized inputs, malicious parsing cases, and evidence manipulation.
Calibration and error analysis	Precision/recall and confidence reliability by task and subgroup; thresholds selected on validation data, evaluated on held-out data.
Regression and reproducibility	Pin code, policy, baseline, model, prompts, and normalization; record nondeterminism; compare candidate versions before release.
Recovery and idempotency	Crash after each durable stage; duplicate queue delivery; partial provider failure; restore checkpoints and verify no unauthorized or duplicate release.
Privacy/security tests	Cross-tenant access, secret leakage, log redaction, destination restrictions, deletion paths, egress controls, and privilege escalation.
Performance/load tests	Defined input mix and concurrency; p50/p95/p99 latency, throughput, memory, queue delay, provider limits, and cost per completed run.
Red teaming and human review	Independent challenge of policy/evaluator assumptions; reviewer disagreement, escalation, fatigue, and exception abuse.

7.21.2 Metrics and acceptance methodology

For a defect detector, precision is $TP/(TP+FP)$, recall is $TP/(TP+FN)$, and false-positive rate is $FP/(FP+TN)$, against an adjudicated reference. Report denominators, confidence intervals, defect prevalence, and sampling method. A low review rate alone is not evidence of quality; it can reflect missed defects. A high block rate can indicate poor inputs, misconfiguration, or an overly aggressive detector.

Measure release-integrity violations, duplicate deliveries, unhandled errors, unsupported-input rate, mandatory-check completion, reviewer overturn rate, and time to resolve holds. Proposed engineering gates include zero observed unauthorized releases in the defined test suite and complete policy/input binding for every emitted decision. These are test expectations, not claims of a zero-risk production system.

For semantic evaluators, compare with simple rules and a single-model baseline before adding ensembles. Use independently adjudicated examples and hold out customer/time slices where practical. Human review is itself fallible; provide clear rubrics, conflict resolution, and feedback mechanisms. Research on interactive machine

learning emphasizes the importance of effective human participation rather than treating people as perfect labels (Amershi et al., 2014).

Define product SLOs only after workload measurements. Batch ingestion can tolerate seconds or minutes where a runtime gate cannot. Separate API availability, queue delay, evaluation completion, and delivery success. Publish detector limitations by language, format, and defect class. A general “99% accurate” claim without those conditions is not meaningful.

7.22 Risks and limitations

Table 23. Principal risks and mitigations. Mitigation reduces exposure; it does not eliminate residual risk.

Risk	Consequence	Mitigation and residual limitation
False confidence	Customers interpret pass as universal safety or truth.	Bind decisions to use/coverage; show unknowns and expiry. Residual unseen defects remain possible.
Imperfect semantic evaluation	Hallucinated rationale or missed contradiction causes wrong disposition.	Evidence spans, structured output, calibrated thresholds, review, and benchmark comparisons. No universal semantic guarantee.
Ambiguous fitness criteria	Different reviewers disagree on acceptability.	Explicit task rubric, policy owner, examples, and exception process. Some ambiguity remains a business judgment.
Customer misconfiguration	Wrong baseline, destination, or thresholds weakens controls.	Safe defaults, policy linting, shadow validation, version review, and change audit. Customer intent cannot be fully inferred.
Model dependency	Outage, price change, or behavior drift interrupts evaluation.	Pin versions where possible, regression tests, budget caps, approved fallback. Mandatory unavailable checks hold.
Adversarial data	Input manipulates evaluator or exploits parser.	Isolation, no evaluator tools, local screening, adversarial tests, bounded formats. Detection alone cannot secure the whole pipeline.
Scaling costs	Long documents, retries, or pairwise checks destroy margins.	Quotas, bounded algorithms, selective inference, metering, and explicit overage consent. Some workloads require higher pricing.
Regulatory uncertainty	Product or contractual obligations change.	Maintain applicability register and legal review; avoid compliance guarantees. Regional differences persist.
Security incident	Sensitive content or tenant data is exposed.	Minimize retention, isolate tenants, restrict access, test recovery, maintain incident response. Prevention is not absolute.
Incumbent response	Bundled intake features reduce perceived value.	Focus on a narrow workflow and verified outcomes; integrate commodity checks. Pricing pressure remains.
Limited integrations	Customers cannot reach value without custom work.	One excellent generic API and demand-led connectors. Decline uneconomic bespoke scope.
Long sales cycles	Enterprise opportunities consume runway.	Engineering-led SME beachhead, paid pilots, annual minimums for private deployments. Some qualified buyers will remain unsuitable initially.

Risk	Consequence	Mitigation and residual limitation
Category education	Buyers do not allocate a separate assurance budget.	Sell a specific ingestion problem and outcome, not terminology. Category adoption is uncertain.
Framework/dependency maintenance	Broad agent stack creates update and compatibility burden.	Minimal allowlist, pinned dependencies, adapter contracts, and replacement options. Source reuse still requires ownership.

The most important product limitation is that input assurance cannot guarantee downstream behavior. A permitted, high-quality input can still produce an incorrect answer or unsafe tool action. Customers must retain output evaluation, access control, model-risk management, and application safeguards appropriate to the task. AIDAP should make this boundary precise without diluting responsibility for its own gate.

7.23 Commercial KPIs

Table 24. Commercial and product KPI definitions. Targets should be set after an initial cohort establishes realistic baselines.

Metric	Definition and management use
MRR / ARR	Recurring subscription revenue; annualize consistently. Separate services, taxes, and pass-through usage so growth is interpretable.
Gross margin	Revenue less consistently allocated delivery costs. Inspect by tier and workload, including support.
CAC and payback	Acquisition spend and attributable sales effort per new paying customer; divide by monthly contribution for payback. Founder sales time is not free.
LTV	Cohort-based contribution estimate only after retention data exists. Avoid extrapolating a few months of low churn indefinitely.
Logo/revenue churn	Lost customers and recurring revenue in the starting cohort; distinguish voluntary churn, failed payments, and project completion.
NRR / expansion revenue	Starting-cohort revenue after expansion, contraction, and churn. Growth from new customers is excluded.
Activation	A project completes a real validation, resolves a finding, and enforces an approved delivery within a defined onboarding window.
Conversion	Qualified trials or paid pilots that become retained subscriptions; report denominators and segment.
Validation runs / active assets	Completed billable runs and assets used in recurring workflows. Raw volume alone is not customer value.
Decision distribution	Pass/review/quarantine/block by policy and asset class, with unknown/error states separately tracked.
False-positive/negative rates	Adjudicated detector errors using representative samples, including unflagged inputs. Do not infer missed defects solely from complaints.
Time saved	Customer-measured review/investigation time before and after comparable workflows; account for new review burden.
Support burden	Delivery-support minutes, escalations, and custom work per account and tier.
Compute cost per validation	Direct compute/inference/storage cost by workload, size, and evaluator depth; include retries and failed work.

Metric	Definition and management use
Expansion and retained use	Additional projects, assets, and policies within existing accounts; distinguish durable adoption from temporary batch spikes.

Initial management gates can be modest and explicit: demonstrate recurring paid use, improve activation time across successive pilots, keep direct support within the tier's modeled envelope, and move mature cohorts toward at least 70% gross margin. The 70% figure is a business objective, not an observed result or universal SaaS benchmark. Do not scale acquisition while activation is poor or support makes low-tier contribution negative.

Pair commercial metrics with safety and reliability metrics. A conversion experiment must not weaken mandatory controls; a cost-reduction experiment must not silently skip checks. Review margin, detector error, customer value, and release integrity together, because optimizing any one alone can undermine the product.

7.24 Long-term strategy

The strongest expansion path is deeper **task-specific assurance APIs and policy packs**, followed by controlled assurance of retrieved context and agent memory. These reuse the same artifact, profile, decision, review, and evidence model. They can increase recurring value without requiring AIDAP to become a general agent runtime or a broad governance platform.

AI/agent evaluation and regression is a logical supporting capability when customers need to test whether an input-policy change improves downstream behavior. It should initially connect to existing evaluation tools. A privacy/safety gateway becomes attractive only after detector quality, latency, availability, and operational staffing support an inline commitment. Enterprise policy distribution and governance evidence export are natural later features; a full governance suite is a different business.

Industry-specific packs should follow paid design partnerships with domain experts and permissioned evaluation data. A generic financial or clinical label is insufficient. Each pack needs a defined use, explicit exclusions, measured failure modes, maintenance ownership, and a commercially viable update process. Sector expansion should be stopped if specialized support consumes the margin it creates.

Maintain a stable assurance contract and portable evidence format. Customers should be able to export policies and decisions, replace model providers, and integrate established checks. This reduces adoption risk and forces AIDAP's retention to depend on maintained quality and operational value. The long-term product should remain recognizable as the place where information is assessed and authorized for a specific AI use.

8. Conclusion

AIDAP is technically plausible as a focused commercial application built around selected SLAI implementations. The existing quality, privacy, safety, configuration, and checkpoint code provides a meaningful starting point. It does not constitute a finished multi-tenant service, a general semantic truth engine, or independently validated assurance. Production viability depends on hardening the execution boundary and building the product infrastructure identified in Sections 7.4-7.6.

The commercial opportunity is a repeatable intake-control workflow for teams that already operate AI features and repeatedly change their inputs. The market is competitive and converging: data-quality, observability, evaluation, and guardrail vendors increasingly overlap. AIDAP should therefore avoid claiming an empty category and earn differentiation through intended-use policies, explicit uncertainty, reliable release enforcement, useful review evidence, and manageable total cost.

Affordability and scalability are compatible when the product remains bounded. Deterministic-first processing, selective inference, asynchronous work, usage caps, and a small supported integration surface can keep costs aligned with subscription revenue. Private deployments and specialized domains require minimum contract values and disciplined scope. Pricing and willingness to pay remain hypotheses until paid cohorts demonstrate recurring value and acceptable support burden.

Long-term defensibility is possible through validated task profiles, customer-specific policy history, permitted feedback datasets, and dependable integrations. It is not guaranteed by multi-agent architecture, model independence, or a category name. The practical path is to prove a narrow gate, preserve evidence of its limits, and expand only where measured customer outcomes justify the additional complexity.

9. Recommendations

Table 25. Prioritized recommendations. Implementation feasibility assumes a small technical company with limited specialist capacity.

Rank	Recommendation	Business impact / feasibility
1	Position AIDAP as AI Data Assurance / AI Input Assurance, expressed through a concrete intake-release workflow.	High impact; immediate. Avoid generic chatbot, observability, or evaluator positioning.
2	Target engineering-led SaaS and RAG teams with recurring text/metadata ingestion and a controllable release boundary.	High impact; feasible through focused discovery and paid pilots.
3	Build an MVP for metadata, schema, freshness, duplicates, configured consistency, tested privacy checks, and auditable decisions.	High impact; technically bounded, subject to hardening.
4	Keep AIDAP policy/evidence/tenancy independent of SLAI through typed adapters; use only necessary agents.	High impact; moderate effort and strong maintenance benefit.
5	Use capped subscription plus usage; test €39/€149/€499 tiers and paid pilot pricing rather than presenting them as validated.	High impact; operationally feasible once metering and billing are reliable.
6	Launch a single-region SaaS with optional approved model APIs; support private deployments selectively at annual minimums.	High impact; avoids premature deployment fragmentation.
7	Compete on intended-use decisions, evidence, review, and economics; integrate existing quality/evaluation tools.	High impact; requires credible demonstrations, not a unique-feature claim.
8	Prioritize typed failure handling, release integrity, baseline approval, tenant isolation, bounded execution, and recovery tests.	Essential technical gate; precedes broad feature work.
9	Defer autonomous repair, continuous policy learning, universal semantic validation, OCR breadth, a connector catalog, and full governance.	High feasibility; protects runway and product clarity.
10	Use twelve months to establish paid repeatability, self-service activation, low support burden, and contribution margin.	High impact; contingent on real customer access and disciplined scope.
11	Expand into validated task-specific packs, retrieval/context fitness, and agent-memory assurance before broader platform ambitions.	Medium-term impact; depends on datasets, measured quality, and retained demand.

The first ninety days should reconcile the SLAI release manifest, define the AIDAP contracts and state machine, isolate the minimum dependency set, and test representative quality/privacy paths. In parallel, secure sanitized customer examples and a small number of paid design partners. The output should be a working, bounded gate with evidence - not a broad demonstration that relies on unimplemented semantic capabilities.

Months four through six should prove enforcement, review ownership, recovery, and pilot value. Months seven through nine should productize accounts, caps, billing, deletion, monitoring, and one integration. Months ten through twelve should improve retention and unit economics, publish measured limitations, and evaluate one advanced fitness profile. Enterprise expansion should depend on support capacity and contract economics rather than the prestige of a prospect.

Continue investment when recurring paid use, validated detector performance, and manageable direct cost reinforce one another. Narrow or reposition when value depends on bespoke consulting or unvalidated semantic claims. These gates should govern expansion and protect affordability.

10. References

The following references use an author-date style consistent with APA conventions. Undated vendor and institutional web pages were retrieved on 17 September 2026. Vendor descriptions and prices are time-sensitive. Repository references identify the inspected source baseline; implementation evidence is distinct from independent runtime validation.

- Acceldata. (n.d.). *Enterprise data observability platform*. Retrieved September 17, 2026, from <https://www.acceldata.io/>
- Amershi, S., Cakmak, M., Knox, W. B., & Kulesza, T. (2014). *Power to the people: The role of humans in interactive machine learning*. *AI Magazine*, 35(4), 105-120.
<https://www.microsoft.com/en-us/research/publication/power-to-the-people-the-role-of-humans-in-interactive-machine-learning/>
- Anomalo. (n.d.). *Data quality and unstructured data monitoring*. Retrieved September 17, 2026, from <https://www.anomalo.com/>
- Anthropic. (n.d.). *Pricing*. Retrieved September 17, 2026, from <https://platform.claude.com/docs/en/about-claude/pricing>
- Arize. (n.d.). *Pricing*. Retrieved September 17, 2026, from <https://arize.com/pricing/>
- Bigeye. (n.d.). *Data observability and AI trust*. Retrieved September 17, 2026, from <https://www.bigeye.com/>
- Chalapathy, R., & Chawla, S. (2019). *Deep learning for anomaly detection: A survey*. arXiv. [Preprint].
<https://arxiv.org/abs/1901.03407>
- Cleanlab. (2026, January 28). *Handshake acquires Cleanlab*.
<https://cleanlab.ai/blog/handshake-acquires-cleanlab/>
- Datadog. (2025). *Datadog acquires Metaplane*.
<https://www.datadoghq.com/blog/datadog-acquires-metaplane/>
- Es, S., James, J., Espinosa Anke, L., & Schockaert, S. (2024). *RAGAs: Automated evaluation of retrieval augmented generation*. Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations (pp. 150-158). Association for Computational Linguistics.
<https://aclanthology.org/2024.eacl-demo.16/>
- European Commission. (n.d.-a). *AI Act*. Retrieved September 17, 2026, from <https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai>
- European Commission. (n.d.-b). *Cyber Resilience Act*. Retrieved September 17, 2026, from <https://digital-strategy.ec.europa.eu/en/policies/cyber-resilience-act>
- European Commission. (n.d.-c). *NIS2 Directive: Securing network and information systems*. Retrieved September 17, 2026, from <https://digital-strategy.ec.europa.eu/en/policies/nis2-directive>
- European Data Protection Board [EDPB]. (n.d.-a). *Data protection basics*. Retrieved September 17, 2026, from https://www.edpb.europa.eu/sme/learn-the-basics/data-protection-basics_en

- European Data Protection Board [EDPB]. (n.d.-b). *Secure personal data*. Retrieved September 17, 2026, from https://www.edpb.europa.eu/sme/be-compliant/secure-personal-data_en
- Fiddler. (n.d.-a). *AI control plane*. Retrieved September 17, 2026, from <https://www.fiddler.ai/>
- Fiddler. (n.d.-b). *Pricing*. Retrieved September 17, 2026, from <https://www.fiddler.ai/pricing>
- Galileo. (n.d.). *Pricing*. Retrieved September 17, 2026, from <https://galileo.ai/pricing>
- Gebbru, T., Morgenstern, J., Vecchione, B., Vaughan, J. W., Wallach, H., Daumé III, H., & Crawford, K. (2021). *Datasheets for datasets*. Communications of the ACM, 64(12), 86-92. Author manuscript. <https://arxiv.org/abs/1803.09010>
- Giskard. (n.d.). *Pricing*. Retrieved September 17, 2026, from <https://www.giskard.ai/pricing>
- Great Expectations. (n.d.). *Pricing*. Retrieved September 17, 2026, from <https://greatexpectations.io/pricing/>
- Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). *Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection*. arXiv. Research manuscript. <https://arxiv.org/abs/2302.12173>
- Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). *On calibration of modern neural networks*. Proceedings of Machine Learning Research, 70, 1321-1330. <https://proceedings.mlr.press/v70/guo17a.html>
- Langfuse. (n.d.). *Pricing*. Retrieved September 17, 2026, from <https://langfuse.com/pricing>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). *Retrieval-augmented generation for knowledge-intensive NLP tasks*. Advances in Neural Information Processing Systems, 33. Author manuscript. <https://arxiv.org/abs/2005.11401>
- Metaplane. (n.d.). *Data observability*. Retrieved September 17, 2026, from <https://www.metaplane.dev/>
- Monte Carlo. (n.d.-a). *Agent trust and data observability*. Retrieved September 17, 2026, from <https://montecarlo.ai/>
- Monte Carlo. (n.d.-b). *Request pricing*. Retrieved September 17, 2026, from <https://montecarlo.ai/request-for-pricing/>
- National Institute of Standards and Technology [NIST]. (2023). *AI Risk Management Framework (AI RMF 1.0)*. <https://www.nist.gov/itl/ai-risk-management-framework>
- Northcutt, C., Jiang, L., & Chuang, I. (2021). *Confident learning: Estimating uncertainty in dataset labels*. Journal of Artificial Intelligence Research, 70, 1373-1411. Author manuscript. <https://arxiv.org/abs/1911.00068>

- Patronus AI. (n.d.). *Research and Digital World Models*. Retrieved September 17, 2026, from <https://www.patronus.ai/>
- Soda. (n.d.). *Pricing*. Retrieved September 17, 2026, from <https://soda.io/pricing>
- The-Outsider-97. (2026a). *SLAI [Source code, inspected baseline for SLAI v2.3]*. GitHub. Commit 265e3407a1deffe1d785215a67314b2da43b2ab9. Relevant source paths are listed in Table 5. <https://github.com/The-Outsider-97/SLAI/tree/265e3407a1deffe1d785215a67314b2da43b2ab9>
- The-Outsider-97. (2026b). *AIDAP [Architecture and application scaffold]*. GitHub. Snapshot 1bcfee4f053d84bfa4c5e45a3b422d9913648533. <https://github.com/The-Outsider-97/AIDAP/tree/1bcfee4f053d84bfa4c5e45a3b422d9913648533>
- Wang, R. Y., & Strong, D. M. (1996). *Beyond accuracy: What data quality means to data consumers*. Journal of Management Information Systems, 12(4), 5-33. https://web.mit.edu/tdqm/www/tdqmpub/beyondaccuracy_files/beyondaccuracy.html
- World Wide Web Consortium [W3C]. (2013). *PROV-Overview: An overview of the PROV family of documents*. <https://www.w3.org/TR/prov-overview/>
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. Advances in Neural Information Processing Systems, 36, Datasets and Benchmarks Track. Author manuscript. <https://arxiv.org/abs/2306.05685>